

低リソース無線環境における長周期同期を用いた、幼児教育向けの音・物理演出連動型輪唱システム

情報工学科 2 年

25G1051

近藤 巧望

—— チーム 17 ——

関屋 咲希 (24G1084) 岩波 晃士 (24G1020)

須磨 奏斗 (24G1080) 木村 海士 (25G1045)

仲里 偉武輝 (25G1090)

2026 年 7 月 16 日

1 はじめに

近年、教育現場において ICT(Information and Communication Technology) や EdTech(Education Technology) の活用が進んでおり、幼児期における音楽活動は、身体的・情緒的・社会的な発達を促す重要な役割を果たすことが知られている。茂野 [1] による乳幼児のリズム同期に関する発達過程の研究によれば、3~4 歳児の段階では一定のリズムに自律的に同期することは困難であり、5~6 歳であってもその能力は十分に発達していない。幼児は周囲の音や動きに「共振」することから表現を始め、他者の意識的な「模倣」を繰り返すステップを経て、初めて正確なリズム同期へと発達していくとされる [1]。このような集団でのリズム同期の獲得は、単なる音楽スキルの向上に留まらず、他者との関係性の構築や社会性の発達にも寄与することが示唆されており、保育・教育現場における本質的な役割をも担っている。更に高橋・篠田 [2] らの研究では、幼児がリズムパターンを記憶・再生する際、演奏者の手元や人形の動きといった「視覚的運動情報」が同時に提示されることで、再生を用意にする強い外敵援助効果(再生の容易化)をもたらすことが実証されている。したがって、音(聴覚情報)だけでなく、音楽と連動して動く人形(視覚的演出)を組み合わせた多感覚的な自動演奏システムを導入し、子どもたちが音や動きを互いに模倣し合える環境を提供することは、幼児のリズム発達及び社会性の育成において極めて有効であると考えられる。このような多感覚演奏システムを教育現場へ広く普及かつ実用化するためには、導入コストや保守性の観点から安価かつ省スペースな汎用マイコン(Arduino 等)を複数台用いた分散配置型のシステム構成が望ましい。しかし、多感覚的な自動演奏システムを、複数の独立したマイコンと Wi-Fi 通信を用いて構築する場合、ネットワーク同期において極めてシビアなリアルタイム性能が要求される。各デバイスが独立した処理を行いながら精密にタイミングを合わせる必要があるシステムでは、わずか数ミリ秒のジッター(タイミングの揺らぎ)が致命的な音楽的破綻を招くことが先行研究でも指摘されている [3][4]。特に不安定な無線ネットワーク環境化においては、サーバからの同期信号を基にクライアント間でいかに精密な同期を維持・確立するかが技術的課題となる。もし、通信ジッターに抛って音と人形の動きの間、あるいはデバイス間で数十ミリ秒以上のズレが生じると、幼児は音楽と人形の動きの因果関係を正しく認識できず、模倣行動が困難となる。これは、高橋ら [2] の研究で示されたリズム知覚への「内的クロック」の適合を阻害し、発達途上にある子どもたちのテンポ間をかえって混乱させ、知的・教育システムとしての有効性を損なうという問題へと直結する。そこで本プロジェクトでは、サーバ手動の長周期同期ストリーミングを用いてネットワーク上の通信負荷とパケット送信回数を最適化することで、Wi-Fi 通信用におけるジッター発生を極限まで抑制し、複数台の Arduino によるズレのない自動演奏および物理演出を実現することを目的とした。具体的には、制約の多い低リソースなハードウェアや不安定な通信環境下であっても、サーバからのデータ転送の周期設計とクライアント側での自律的なタイミング制御を組み合わせることで、人間のリズム知覚(特に発達段階にある幼児の知覚)では検知できないレベルの高精度な同期制御が可能であることを実証する。これにより、教育現場における実用的な多感覚演奏・協調学習システムの技術的基盤を確立することを目指し、開発を行った。

現在、ネットワークを介したデバイス間の同期手法として、NTP(Network Time Protocol) その軽量版である SNTP(Simple Network Time Protocol) が広く用いられている [5]. しかし、これらは一般的な時刻合わせを主目的としており、無線 LAN 環境化では非対称な通信遅延やパケットの揺らぎによって数十ミリ秒以上の誤差が生じやすい. そのため、自動演奏に於いて要求されるシビアな同期精度を満たすことは困難である. より高精度な同期規格として、マイクロ秒単位の同期が可能な PTP(Precision Time Protocol)[6] があるが、これは有線 LAN 環境や、専用のハードウェアタイムスタンプ機能を持つ高度なネットワーク機器を前提として設計されている. そのため、Arduino のような低リソースなマイコンと汎用 Wi-Fi モジュールを組み合わせた安価な構成においては、処理負荷の観点から実装が非現実的である. さらに、これらの既存技術はいずれも高頻度に通信を行い、常にネットワーク経由で誤差を修正するというアプローチを取るため、無線チャネルの輻輳やマイコンの割り込み処理によるジッターの悪化を根本的に解決することができない. これに対し、本プロジェクトの最大の独自性は、同期の維持を高ヒントなネットワーク通信に依存せず、長周期の UDP ブロードキャスト通信とクライアント側での自律的なタイミング補間の組み合わせによって解決する点にある. 具体的には、サーバは、頻繁な時刻同期を行うのではなく、現在の BPM に基づいて算出した 1 小節ごとのタイミングでのみ、小節番号を送信する. この際、各クライアントへ個別に送信するのではなく、UDP を用いたローカルブロードキャストを採用することで、サーバからの送信時間差を完全に排除している. 更に、通信データに関して小節番号と BPM 値を閾値で区別可能な 1Byte のバイナリデータに圧縮することで、通信負荷を最小化し、無線 LAN 環境下でのパケットロスやジッターの発生を抑制している. また、クライアント側の処理及びエラーハンドリングにおいても独自のアプローチを取る. 小節番号や BPM 値を受信した各クライアントは、受信データから書く音符の長さを算出し、次の小節同期信号を受信するまでの間、マイコンの内部タイマーを用いて自律的かつ連続的に演奏タイミングを制御する. この際、パケットロスが生じた場合二、再送処理による遅延データの受信が致命的な楽曲のズレを引き起こすことを防ぐため、あえて再送機構を持たない UDP 通信を採用している. 更に、アプリケーション層での再送要求も意図的に排除している. パケットラジスは該当小節の演奏を行わず、次の小節信号で復帰させることで、楽曲全体のハーモニー崩壊を防ぐという、音楽的特性を活かしたフェイルセーフな設計となっている. 既存技術がネットワークに依存した密な同期を目指して破綻していたのに対し、本手法は通信を最小化し、クライアント側での自律的なタイミング制御を組み合わせることで、低リソースなマイコンと不安定な無線 LAN 環境下においても、幼児のリズム知覚では検知できないレベルの高精度な同期制御を実現することが可能となり、本プロジェクトの独自性と優位性を示している.

2 作成したシステムの概要

本システムは、1 台のサーバと 4 台のクライアントから構成される分散型の自動演奏システムである. サーバとクライアントにはいずれも Arduino を用いる. サーバは楽曲全体のテンポを管理する指揮者としての役割を担い、各クライアントはピアノ、トロンボーン、カスタネット、バイオリンのいずれか 1 つの楽器を担当する奏者として動作する. サーバとクライアントは同一の Wi-Fi

ネットワークに接続され、サーバはテンポと小節番号を UDP ローカルブロードキャストによって全クライアントへ一斉送信する。

本システムは通信・同期機能、楽器音源の生成・再生機能、人形の物理演出機能の3つの機能から構成される。通信・同期機能は、サーバが送信する小節番号とテンポをクライアントが受信し、演奏のタイミングを制御する機能である。楽器音源の生成・再生機能は、クライアントが受信した小節番号から演奏する音符を決定し、シリアル通信を介して PC 上の Processing へ送信することで音源を再生する機能である。人形の物理演出機能は、クライアントが受信した小節番号に基づいてサーボモータを駆動し、演奏に同期した人形の動作を行う機能である。報告者は、このうち楽器音源の生成・再生機能、および人形の物理的な製作を担当した。本節では、まずシステム全体の構成と信号の流れを示し、続いて各機能について説明する。

2.1 システム全体の構成

2.2 システム全体の構成

本システムの全体構成を図1に示す。サーバと4台のクライアントは同一の Wi-Fi ネットワークに接続される。サーバは楽曲のテンポと演奏中の小節番号を管理し、これらを UDP ローカルブロードキャストによって全クライアントへ一斉に送信する。各クライアントには演奏する楽曲の譜面データがあらかじめ保持されており、受信した小節番号に対応する音符を譜面データから読み出す。

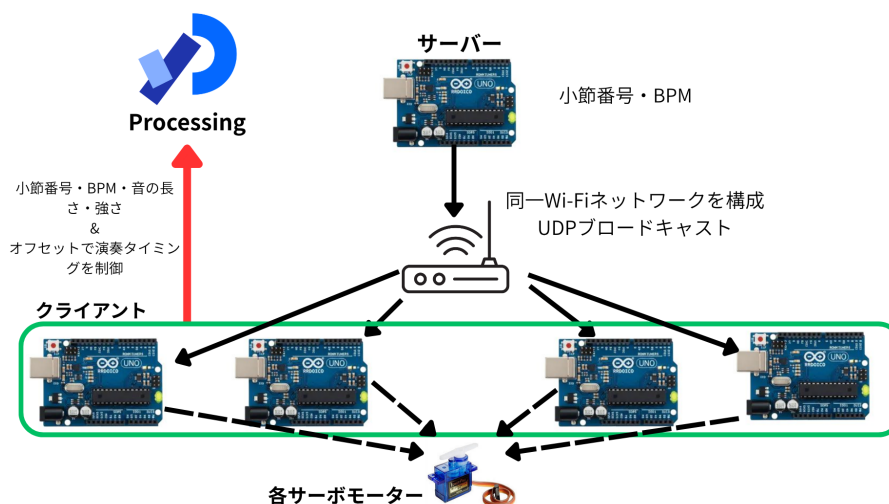


図1 システム全体の構成

各クライアントが担当する楽器の発音は、クライアントに接続された PC 上の Processing が行う。各クライアントは、譜面データから読み出した音符の周波数、発音する長さ、強さ、およびテンポをシリアル通信によって Processing へ送信する。Processing は受信した情報をもとに楽器音源を合成し、スピーカから再生する。このように、サーバからクライアントへは Wi-Fi を介して小節

番号とテンポが送信され、各クライアントから Processing へはシリアル通信を介して個々の音符の情報が送信されるという二段階の信号の流れによって、システム全体の演奏が実現される。また、各クライアントは受信した小節番号に基づいてサーボモータを駆動し、演奏に同期した人形の動作を行う。

2.3 通信・同期機能

通信・同期機能は、サーバが送信する演奏位置とテンポの情報をクライアントが受信し、各クライアントの演奏のタイミングを制御する機能である。同期通信の構成を図2に示す。サーバには2つのスイッチが接続されており、一方を押すとテンポが増加し、他方を押すとテンポが減少する。これにより、演奏中にテンポをリアルタイムで変更できる。サーバは、現在のテンポと演奏中の小節番号を1バイトのデータとしてUDPでブロードキャストする。クライアントは受信したデータの値によってその種別を判別し、一定値以上であればテンポ、それ未満であれば小節番号として扱う。

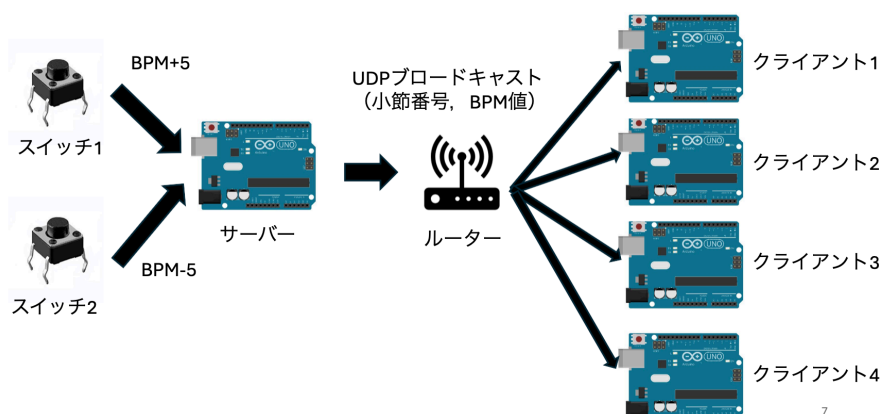


図2 同期通信の構成

テンポを受信したクライアントは、そのテンポから基準となる4分音符の長さを算出し、以降の演奏の時間基準とする。小節番号を受信したクライアントは、あらかじめ保持している譜面データからその小節に含まれる音符を先頭から順に読み出し、各音符の相対的な長さと基準音符長から求めた間隔で、音符の情報を Processing へ順次送信する。これにより、サーバが小節単位の位置情報のみを送信し、各クライアントが自身のもつ譜面データに基づいて小節内の演奏を自律的に進める構成となっている。

また、各クライアントは輪唱を実現するためのオフセットを保持しており、受信した小節番号にオフセットを加えることで自身の演奏開始位置をずらす。これにより、同一の譜面を複数のクライアントが時間差で演奏し、輪唱として成立させている。

2.4 音源の生成・再生機能

楽器音源の生成・再生機能は、クライアントから送信された音符の情報をもとに、PC上の Processing が楽器の音を合成して再生する機能である。Processing は、クライアントからシリアル通信で受信した周波数、発音する長さ、強さ、およびテンポを解釈し、指定された周波数の音を、その楽器に応じた音色で、指定された長さだけ発音する。

各楽器の音色は、実際の楽器の音響的な特徴を反映するように設計した。例えば、トロンボーンは音高の明確な持続音を発する楽器であるため、加算合成によってその音色を再現した。一方、カステネットは音高を持たない打楽器であり、特定の周波数帯に集中した打撃音を再現した。これらの音源の具体的な設計と実装については??章で述べる。

2.5 物理演出機能

物理演出機能は、演奏に合わせて楽器を持った人形を動作させ、視覚的な演出を行う機能である。各クライアント Arduino にはサーボモータが接続されており、人形の腕などの可動部を駆動する。クライアントは、受信した小節番号に基づいてその小節で演奏すべき動作のタイミングを求め、音の発生に同期してサーボモータを動かす。これにより、人形が楽器を演奏しているように見せる。

また、各クライアントはテンポの変化を受信した際に、人形に演奏以外の動作を行わせる。これにより、楽曲の進行に合わせた人形の反応を表現する。人形は演奏する楽器に応じて異なる動作を行い、楽器ごとの演奏の様子を表現する。ピアノの場合、両手で鍵盤を叩くような動作を行い、トロンボーンの場合は楽器をベルアップさせて下げる動作を繰り返す。ヴァイオリンは、弓を楽器に当ててさらに口が開くような動作を行い、カステネットは両手でカステネットを叩く動作を行う。カステネットは、サーバー側のテンポ変化が起きると頭が回転するという動作も行う。

3 システム実装

3.1 チーム構成と役割分担

本プロジェクトは、6名(近藤・関屋・岩波・須磨・木村・仲里)のチームで実施した。本システムでは、同期通信機能・楽器音源の生成・再生機能・人形の物理演出機能の3つの機能が必要であり、チームメンバーはそれぞれの担当を分担した。このチームでは、機能ごとにタスクを担当するのではなく、細かい成果物ごとに担当を分担する方式を採用した。特に、Processing や Arduino といったプログラムはハッカソン1の学生が主体となり、ハッカソン2の学生にそのレビュー及び、手の回らない部分の実装を依頼するという形で、チーム内での役割分担を行った。なお、本システムのサーバおよびクライアントはいずれも Arduino で実装されており、主要な関数はサーバ側とクライアント側に分けられる。

[サーバ側]

Bar_control() 関数 小節番号を送信する関数である。前回の送信から一定の間隔が経過するたびに小節番号を1つ進め、その値をUDPでブロードキャスト送信する。小節番号は0から39までの範囲で循環させる。フローチャートを図3に示す。

BPM_control() 関数 テンポを変更する関数である。テンポを増減する2つのスイッチの押下を検出し、上限・下限の範囲内でテンポを5ずつ増減させる。テンポの変更後、最後にスイッチが押されてから一定時間が経過した時点で、変更後のテンポをUDPで送信する。これにより、スイッチを連続して押した場合でも送信が1回にまとめられる。フローチャートを図4に示す。

detectPress() 関数 スwitchの押下を検出する関数である。スイッチの読み値には接点の機械的な振動によるチャタリングが生じるため、読み値が一定時間安定して初めて状態の変化を確定する。これにより、1回の押下が複数回として誤検出されることを防ぐ。フローチャートを図5に示す。

[クライアント側]

Parse_data() 関数 受信データの種別を判別する関数である。サーバから送られる1バイトのデータは小節番号とテンポのいずれかであり、その値が一定値以上であればテンポ、それ未満であれば小節番号として扱う。小節番号の場合は、各クライアントに固有のオフセットを加えて自身の演奏位置を求める。フローチャートを図6に示す。

BPM_update() 関数 テンポから演奏の時間基準を求める関数である。テンポを受信した際に、基準となる4分音符の長さをミリ秒単位で算出し、以降の演奏タイミングの基準とする。フローチャートを図7に示す。

Performance() 関数 演奏を進める関数である。受信した小節番号に対応する音符を、あらかじめ保持している譜面データから先頭から順に読み出す。各音符の周波数、発音する長さ、強さ、およびテンポをシリアル通信でProcessingへ送信し、音符の長さに応じた間隔をおいて次の音符へ進む。フローチャートを図8に示す。

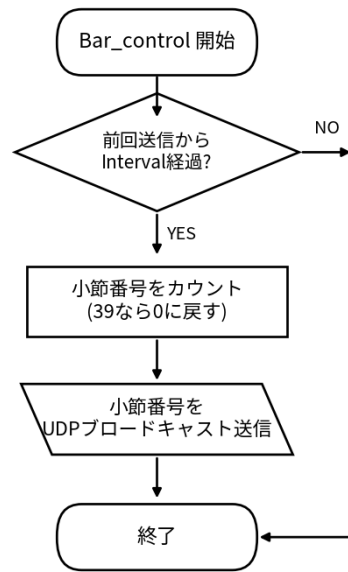


図 3 Bar_control() 関数のフローチャート

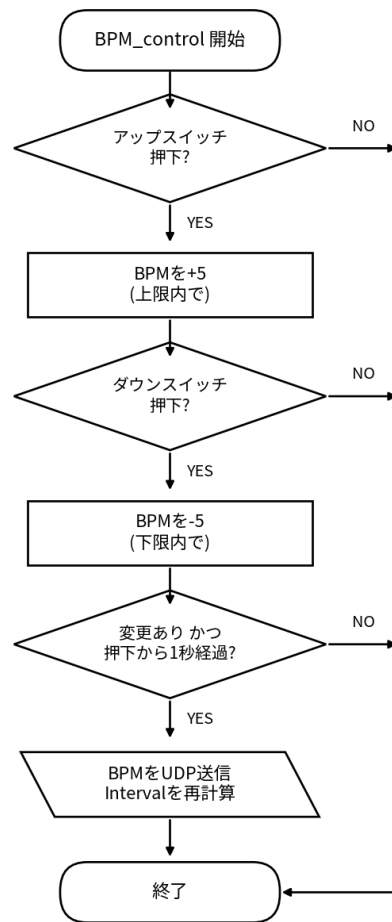


図 4 BPM_control() 関数のフローチャート

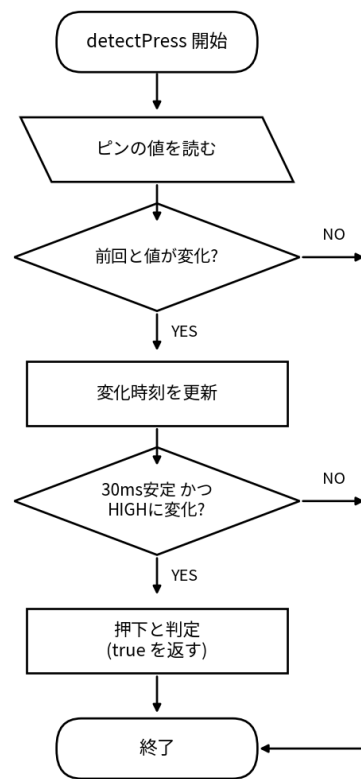


図 5 detectPress() 関数のフローチャート

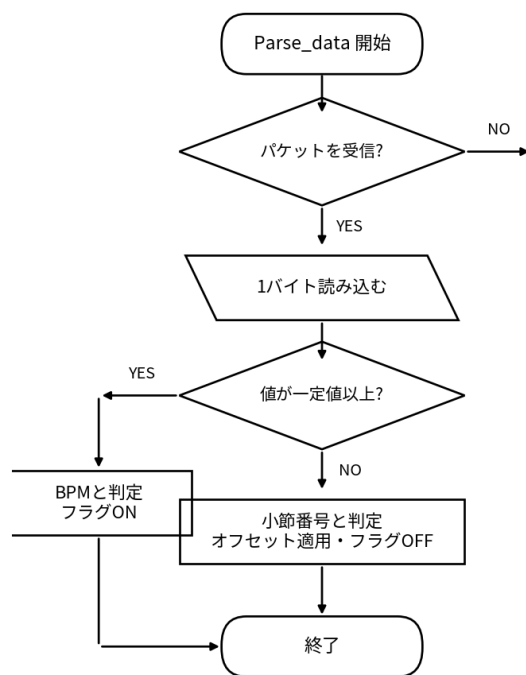


図 6 Parse_data() 関数のフローチャート

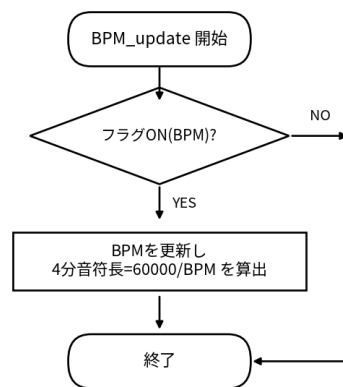


図 7 BPM_update() 関数のフローチャート

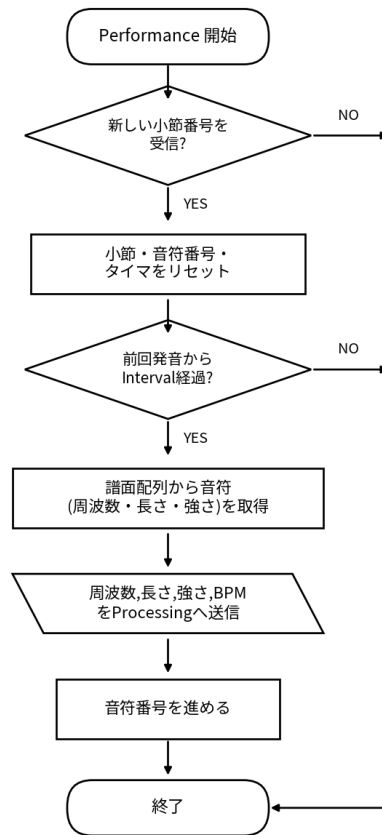


図 8 Performance() 関数のフローチャート

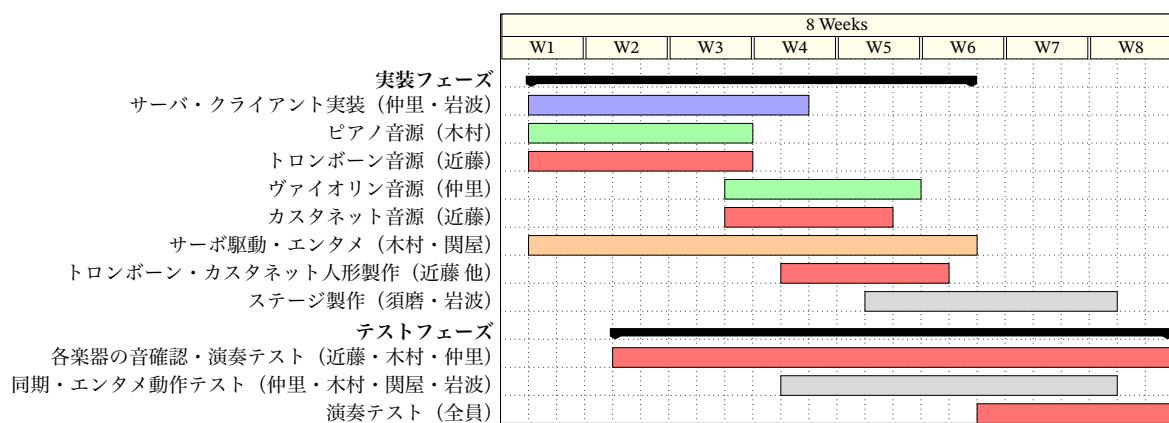


図 9 開発スケジュール

図9に、開発スケジュールを示す。まず、サーバー・クライアントの実装は仲里・岩波が担当し、サーバーの同期通信機能を実装した。次に、Processing 上での楽器音源の生成は、それぞれトロンボーンとカスタネットの音色作成を近藤が担当し、ピアノの音色作成を木村が担当した。ヴァイオリンの音色作成は仲里が担当した。人形のサーボモータ駆動を含むエンタメ部分の実装は木村・関屋が主に担当し、後に人形のサーボモータ接合部の実装は近藤と仲里で手伝った。人形を配置するステージの作成は須磨・岩波が担当した。テストフェーズでは、各楽器の音確認・演奏テストを近藤・木村・仲里が担当し、サーボモータの動作確認テストは木村と関屋が主に担当した。同期テストはサーバとクライアントの実装を行った仲里と岩波が担当し、最終的な演奏テストは全員で実施した。

3.2 報告者の担当範囲と技術的課題

報告者は、トロンボーンおよびカスタネットの音源を Processing 上で実装した。他にも後付の作業として人形作成が加わったが、そこらは物理的な作業であり、報告者の専門分野ではないため、本報告書では主に音源の実装について書く。音源は、クライアントから送信される音符の周波数をもとに、Minim ライブラリを用いて波形を合成することで生成する。

トロンボーンの音源を実装するにあたり、まず単純な波形を用いて音を合成した。しかし、矩形波や単一のオシレータ(一定周期の波形を出力する発振器)による波形では、生成される音は電子的なブザー音のように聞こえ、実際のトロンボーンの声とは大きく異なるものであった。実物のトロンボーンは、基本周波数に加えて多くの倍音を含み、また音の立ち上がりや減衰にも特有の変化を持つ。単純な波形ではこれらの特徴が再現されないため、トロンボーンらしい音色を得ることが課題となった。

カスタネットは、トロンボーンとは異なる種類の課題を持つ。実際のカスタネットの音の波形を図10に示す。図から、打撃の瞬間に振幅が最大となり、その後急速に減衰していること、および波形が正弦波のような規則的な周期を持たず、不規則に振動していることが読み取れる。

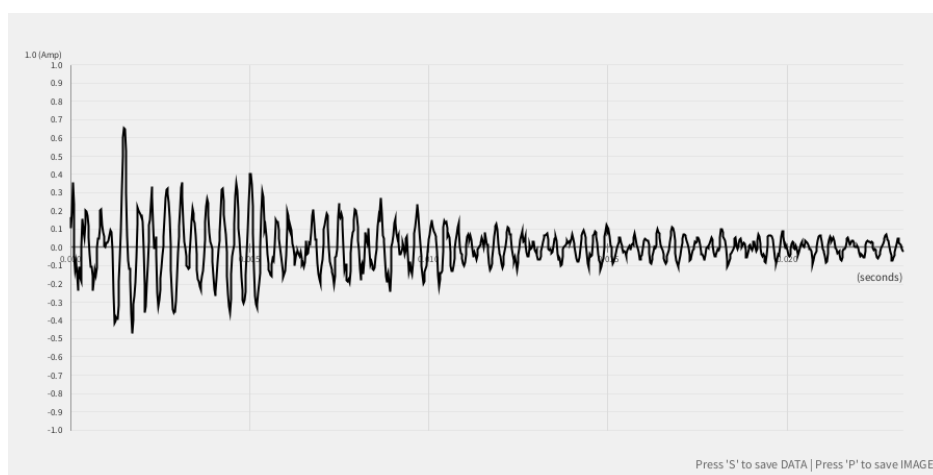


図10 実際のカスタネットの音の波形

一方、オシレータは周期的な波形を出力するため、明確な基本周波数を持ち、音程のある音として知覚される。これは、図 10 に示したカスタネットの不規則な波形とは根本的に異なる。そのため、音程を持たず、不規則な打撃音として聞こえるカスタネットの音を、オシレータを基本とする Minim の枠組みでいかに合成するかが課題となった。

3.3 予備実験と設計値の決定

音源の各パラメータは、実測から一意に定まるものではなく、実際の楽器の音に近い聴感が得られるように調整して決定する必要がある。そこで、Processing 上で合成音の波形および周波数成分を確認しながら、実際の楽器音と聴き比べる予備実験を繰り返し、各設計値を決定した。

トロンボーンの音源では、波形テーブルの倍音構成に加え、音色を整えるためのフィルタおよびエンベロープのパラメータを調整した。波形テーブルには高次の倍音まで含まれており、そのままでは電子的で硬い音となる。そこで、遮断周波数 1200 Hz のローパスフィルタを適用し、高域成分を抑えることで管楽器らしい柔らかい音色に近づけた。また、音の立ち上がりが急峻すぎると機械的な印象となるため、立ち上がり時間を 0.08 秒、減衰時間を 0.05 秒、持続レベルを 0.8、余韻を 0.3 秒とするエンベロープを設定し、管楽器の発音に近い緩やかな立ち上がりとした。さらに、発音直後に基準周波数の 0.96 倍から 0.12 秒かけて基準周波数へ移行させることで、金管楽器にみられる発音時のわずかなピッチの揺れを再現した。設計値を表 1 に示す。これらの値は合成音を試聴しながら段階的に決定した。

表 1 トロンボーン音源の設計値

パラメータ	設計値
ローパスフィルタ遮断周波数	1200 Hz
フィルタのレゾナンス	0.2
立ち上がり時間 (Attack)	0.08 秒
減衰時間 (Decay)	0.05 秒
持続レベル (Sustain)	0.8
余韻 (Release)	0.3 秒
発音時のピッチ変化	基準の 0.96 倍 → 基準 (0.12 秒)

カスタネットの音源では、打撃音を構成する 3 つの成分について、中心周波数、フィルタ特性、および減衰時間を調整した。各成分はノイズを帯域通過フィルタに通して生成するため、フィルタの中心周波数と帯域幅によって音の質感が決まる。木の胴鳴りに相当する主成分の中心周波数を 1000～2600 Hz の範囲に設定し、これを基準として表面の硬い打撃音を約 2500～3600 Hz 高い帯域に、高域の乾いた成分を約 4200～5200 Hz 高い帯域に配置した。各成分の減衰時間は、打撃音の余韻が短く自然に減衰するように設定し、胴鳴り成分の減衰を 0.060 秒とやや長く、表面の打撃音を 0.007 秒、高域成分を 0.018 秒と短くした。いずれの成分も持続レベルを 0 とし、持続音を持たない打楽器の特性を再現した。設計値を表 2 に示す。

さらに、実際のカスタネットが打つたびに異なる音になることを再現するため、各成分の中心周波数に発音ごとに ± 90 Hz の揺らぎを与え、振幅も 0.80～1.05 倍の範囲で変動させた。これらの値は、波形および周波数成分の確認と聴感評価の双方から決定した。

表 2 カスタネット音源の各成分の設計値

成分	中心周波数	レゾナンス	立ち上がり	減衰
表面の硬い打撃音	主成分 + 約 3000 Hz	0.65	0.5 ms	7 ms
木の胴鳴り	1000～2600 Hz	0.85	1 ms	60 ms
高域の乾いた成分	主成分 + 約 4700 Hz	0.55	0.5 ms	18 ms

3.4 その他の実装

また音色作成以外の実装として、人形の物理制作・カスタネット用 client の変更を行った。

人形制作に関しては、楽器の模型を作り、サーボモータで動く人形の腕で楽器を演奏するように段ボールやテープを用いて制作した。

カスタネット用 client の変更に関しては、その他の楽器トロンボーン・ピアノ・ヴァイオリンで共通で使用しているクライアントプログラムである client.ino を、カスタネット用に変更した。client.ino では、音符の周波数・長さ・強さを Processing に送信する処理が行われているが、カスタネットは音程を持たないため、音の高さを表す周波数の情報は本来不要である。そこで、他の楽器が音階 (262～880 Hz) を音符ごとに切り替えていたのに対し、カスタネットでは発音する周波数を固定値の 1800 Hz とした。これにより、どの打撃に対しても一定の帯域を中心とした打撃音が生成される。

また、譜面配列についても、音階を並べたメロディではなく、一定の周波数の打撃と休符を組み合わせた打撃パターンとして構成した。さらに、カスタネットのクライアントは音源の再生のみを担当し、人形の動作を制御する処理は分離した。

3.5 担当箇所以外の実装

3.5.1 チャタリング対策回路

サーバでは、テンポを変更するためのスイッチが用いられている。スイッチには接点の機械的な振動によるチャタリングが生じるため、これを除去する必要がある。チャタリングの除去はソフトウェアで実装することも可能であるが、本システムでは小節番号の通信におけるリアルタイム性を重視し、サーバの処理量を抑えるために、ハードウェアによるチャタリング対策を行う。BPM 変更スイッチのチャタリング対策回路を図 11 に示す。

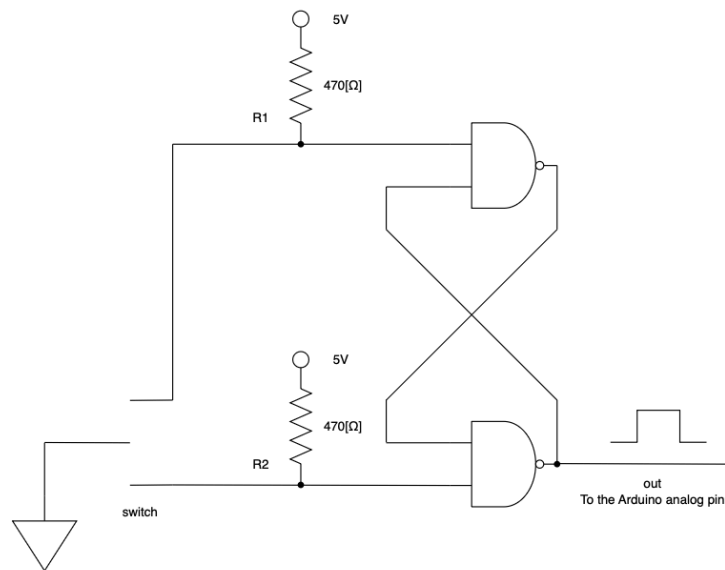


図 11 BPM 変更スイッチのチャタリング対策回路

この回路は、2つの NAND 素子を交差接続したフリップフロップを用いており、スイッチが切り替わると回路の状態が保持される。これにより、接点のバウンスによる複数回の信号変化が出力に現れず、1回の安定した信号が得られる。出力を Arduino のデジタルピンに接続し、`digitalRead` 関数でスイッチの押下を検出する。BPM 変更スイッチはアップ用とダウン用の2つが必要であるため、この回路もそれぞれに対して実装する。

3.5.2 サーボモータの接続

クライアントでは、人形の可動部を駆動するためにサーボモータを用いる。サーボモータと Arduino の接続を図 12 に示す。サーボモータの信号線を Arduino のデジタル出力ピンに接続し、5V を供給して駆動する。受信した演奏情報に応じてサーボを動作させることで、演奏に同期した人形の動きを実現する。図 12 では、サーボモータの電源として Arduino の 5V を使用しているが、複数のサーボモータを同時に駆動する場合は、外部電源を使用することが望ましい。実際に、トロンボーン人形を除いたピアノ・カスタネット・ヴァイオリンの3体はそれぞれ2つのサーボモータを使用しており、Arduino の 5V では電流が不足するため、外部電源を使用している。なお、図中ではサーボモータを D9 ピンに接続しているが、人形の右腕を D9 ピンに、左腕を D11 ピンに、頭や口などは D5 ピンに接続する設計となっており、サーボモータの接続ピンは人形ごとに異なる。

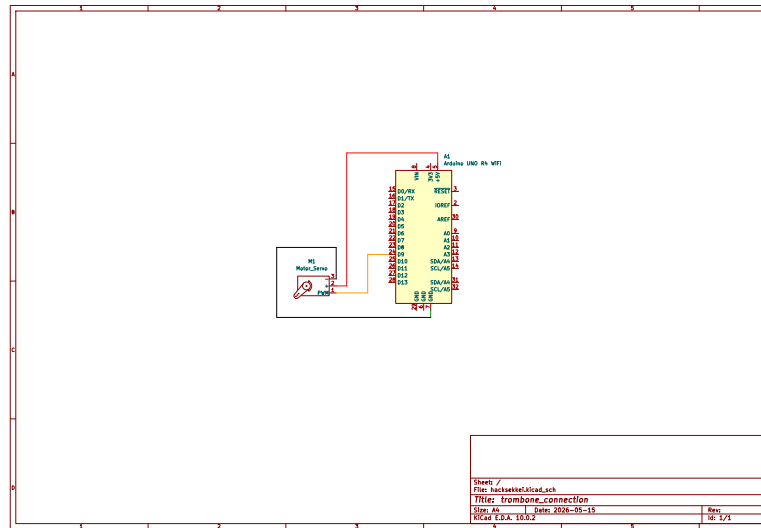


図 12 サーボモータを使用したクライアント部分の回路図

3.6 生成 AI の活用と検証

本開発では、音源解析に関するコードの作成に生成 AI を活用した。一方で、生成 AI が出力する結果は、その処理内容を理解した上で妥当性を検証する必要がある。本節では、トロンボーンの音源設計における生成 AI の活用とその検証について述べる。

トロンボーンの音色を設計するにあたり、実際のトロンボーンの音に含まれる倍音の構成を参考にすることとした。そこで、音源のスペクトルから倍音の振幅を抽出する FFT 解析のコードの作成を生成 AI に依頼し、得られたコードを用いて解析を行った。当初はこの解析結果を倍音の振幅として波形テーブルに用いた。しかし、後の検証において、抽出された周波数がいずれも基本周波数の近傍に集中しており、倍音を正しく抽出できていないことが判明した。原因を確認したところ、生成されたコードは FFT の振幅を単純に大きい順に並べるものであり、スペクトルのピークを検出する処理を含んでいなかった。そのため、倍音のピークではなく、基本周波数の 1 つのピークの近傍にある複数の値を抽出していた。この問題を踏まえ、解析コードにピーク検出処理 (`scipy.signal.find_peaks`) を導入し、ピーク間の最小間隔と卓立度を指定することで、倍音のピークを適切に分離して抽出できるように改善した。この経験から、生成 AI が出力するコードは、処理の内容と結果の妥当性を利用者自身が確認した上で採用する必要があることを確認した。以降の解析では、公式ドキュメントを参照しながら処理内容を確認し、生成結果を検証した上で採用する方針を取った。

カスタネットの音源生成においては、これを踏まえて、生成 AI に対して FFT 解析のコードを作成する際に、ピーク検出処理を含めるように指示した。更に、Processing 上での音色再現についても、減算合成の手法を用いて、FFT 解析の結果をもとに各成分の中心周波数・減衰時間・フィルタ特性を調整するコードを生成 AI に作成させた。導入する以前の音色と比較すると、明らかにより

実際のカスタネットの音に近い音色が得られ、その他の問題が発生しないことを確認し、実際のシステムに採用した。

また、楽器の単体テスト用に波形を出力し、周波数を表示するコードを生成 AI に作成させ、実際の楽器音と合成音の周波数成分を比較することで、設計値の妥当性を検証した。コードの中では、もともと ino ファイルで定義されている周波数をそのまま使用しているので正当性は担保されている。それに加え、評価で用いる自己相関法を用いた、理論周波数との相対誤差を算出するコードと、その結果をグラフ化するコードも生成 AI に作成させ、ゼロ交差法も同様に生成 AI に作成させた。

4 評価方法と結果

4.1 評価方法

報告者が担当した楽器音源について、各楽器の音確認テストと演奏テストの2つを実施した。音確認テストでは客観評価と主観評価の両面から、演奏テストでは発音の完遂性から評価を行った。

4.1.1 音確認テスト

音確認テストでは、Processing によって生成される音が、各楽器の物理的特性に基づく理論的な周波数を正確に再現できているかを検証する。客観評価では、合成音の波形を取得し、その基本周波数を測定して目標とする音高の理論周波数と比較する。基本周波数の測定には、自己相関法およびゼロ交差法を用いた。測定された周波数 f_{meas} と理論周波数 f_{ref} との相対誤差を次式で定義する。

$$\varepsilon = \frac{|f_{\text{meas}} - f_{\text{ref}}|}{f_{\text{ref}}} \times 100 \quad [\%] \quad (1)$$

評価基準は、相対誤差 ε が $\pm 1.0\%$ 以内であることとした。

主観評価では、被験者に各楽器の音を提示し、「意図された楽器を正しく識別できる」および「音の減衰や響きが自然である」の2項目について5段階（5:非常に思う～1:全く思わない）で評価を得た。被験者数を N 、各被験者の評価値を S_i としたとき、平均評価値 S_{avg} を次式で定義する。

$$S_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N S_i \quad (2)$$

評価基準は、平均評価値 S_{avg} が 4.0 以上であることとした。

4.1.2 演奏テスト

演奏テストでは、Arduino による譜面配列の参照から Processing での発音に至るまで、実曲の演奏においてシステムが正常に機能するかを検証する。複数の小節にわたる連続的な演奏を対象と

し、リズムのずれや音の欠落が発生しないかを確認する。評価指標として、正常に発音された音符数の割合を示す発音成功率 R_{success} を次式で定義する。

$$R_{\text{success}} = \frac{\text{正常発音数}}{\text{全音符数}} \times 100 \quad [\%] \quad (3)$$

ここで正常発音数は、譜面配列の音符情報が欠落なく Processing へ送信され、設計通りの周波数で発音された回数を示す。評価基準は、発音成功率 R_{success} が 100% であることとした。

4.2 客観評価の結果

各楽器音源の客観評価の結果を表 3 に示す。また、各楽器について、実際の楽器の音の波形と合成音の波形を取得し、比較を行った。

報告者が担当したトロンボーンでは、目標とする C3（理論周波数 131.0 Hz）に対し、測定された周波数は 131.05 Hz、音高の誤差は +0.3 cent であり、誤差 $\pm 1.0\%$ 以内という基準を満たした。トロンボーンの実際の音の波形を図 13 に、合成音の波形を図 14 に示す。実際の波形は、基本周期ごとに鋭いピークを持つ倍音の豊かな波形を示している。合成音の波形も同様に周期的な波形を示しており、基本周波数が再現できていることが確認できる。一方で、合成音は実際の波形に比べて振幅が小さく、波形の細部の複雑さも少ない。これは、後述するように音源の音量が小さいこと、および倍音構成が実際の楽器と完全には一致していないことによると考えられる。

カスタネットについては、音程を持たない打楽器であるため、周波数の誤差ではなく波形の特性で評価した。カスタネットの実際の音の波形を図 15 に、合成音の波形を図 16 に示す。実際の波形は、打撃の瞬間に振幅が最大となり、その後不規則に振動しながら急速に減衰している。合成音の波形も、立ち上がり直後に振幅が最大となり、約 20 ms で減衰する打撃音特有の波形を示した。ピーク振幅は 0.17 であり、音程を持たない打撃音として妥当な波形が得られたと判断した。

また、参考として、他のメンバーが担当したピアノおよびヴァイオリンについても客観評価を行い、いずれも理論周波数との誤差 $\pm 1.0\%$ 以内を満たした。さらに、すべての楽器について実曲演奏テストを行い、全音符が欠落なく発音され、発音成功率は 100% であった。

表 3 楽器音源の客観評価の結果

楽器	目標周波数	測定結果	判定
ピアノ	C4 (261.6 Hz)	261.67 Hz (+0.3 cent)	達成
トロンボーン	C3 (131.0 Hz)	131.05 Hz (+0.3 cent)	達成
ヴァイオリン	C4 (261.6 Hz)	263.15 Hz (+0.58 cent)	達成
カスタネット	—	約 20 ms で減衰する打撃波形	達成

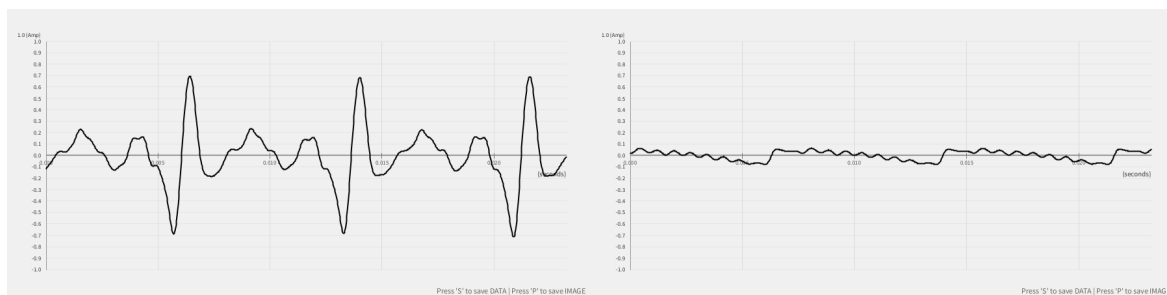


図 13 トロンボーン（実物）の波形

図 14 トロンボーン（合成音）の波形

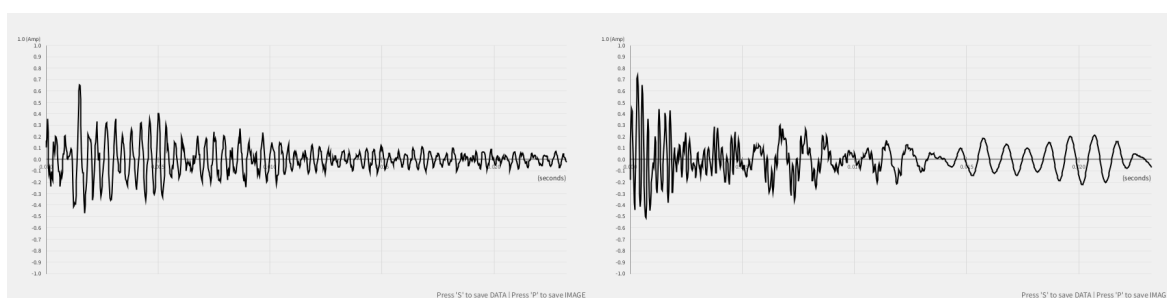


図 15 カスタネット（実物）の波形

図 16 カスタネット（合成音）の波形

4.3 主観評価の結果

32 名を対象に実施したアンケートによる主観評価の結果を表 4 に示す。報告者が担当したトロンボーンは、作成音 3.88、音質 3.97 であり、全楽器の中で比較的高い評価を得た。カスタネットは作成音 3.44、音質 3.81 であった。一方で、いずれの楽器も作成音の評価は基準の 4.0 に達しなかった。これは、合成音の音量が小さいことや、演奏時のサーボモータの駆動音が影響したためと考えられる。音質については、トロンボーンをはじめ複数の楽器で 3.8 以上の評価が得られた。

表 4 楽器音源の主観評価の結果 ($n = 32$, 5 段階)

楽器	作成音	音質
ピアノ	3.56	3.97
トロンボーン	3.88	3.97
ヴァイオリン	3.06	3.22
カスタネット	3.44	3.81

5 考察

5.1 客観評価と主観評価の乖離

報告者が担当したトロンボーンおよびカスタネットは、客観評価においては周波数誤差 $\pm 1.0\%$ 以内という基準を満たした一方で、主観評価では作成音の平均評価値がいずれも基準の 4.0 に達しなかった。この乖離について考察する。

ここで、客観評価と主観評価では測定条件が異なる点に注意する必要がある。客観評価は音源単体で取得した波形に基づくものであり、音源の周波数特性そのものを評価している。これに対し主観評価は、人形の動作を含むシステム全体の演奏を提示して行ったものである。したがって、主観評価には音源そのものの音色だけでなく、演奏環境全体の要因が反映される。

主観評価が伸びなかった要因としては、合成音の音量が実際の演奏環境において十分でなかったこと、および他楽器と比べ、オクターブを一つ下で設定しているため、音色が低く聞こえ、音量が小さく感じられたことが考えられる。特にヴァイオリンは高音域で音色的にも目立つ楽器なので、トロンボーンやカスタネットの音が埋もれてしまい、主観評価において低い評価となったと考えられる。これに関してはピアノでも同様の傾向が見られたため、ヴァイオリンの音が他楽器の音が埋もれる原因となった可能性が高いと思われる。

5.2 トロンボーンの音色成立の要因

トロンボーンの音源では、波形テーブルに設定した倍音構成が、実際のトロンボーンの倍音構造を正確に反映したものではなかった。にもかかわらず、合成音は電子的なブザー音ではなく、管楽器に近い音色として成立し、主観評価においても音質は 3.97 と比較的高い評価を得た。この要因について考察する。

本音源では、オシレータが生成する波形に対し、ローパスフィルタ、ディレイ、および音量エンベロープを直列に適用している。ローパスフィルタは高次の倍音成分を抑制するため、波形テーブルの倍音構成が正確でなくとも、過剰な高域成分が除去され、硬い電子的な質感が緩和される。また、音量エンベロープによる緩やかな立ち上がりと減衰は、管楽器特有の発音の印象を与える。これらのことから、本音源の音色は、波形テーブルの倍音構成そのものよりも、フィルタおよびエンベロープによる後段の処理に強く依存して決定されていたと考えられる。この結果は、オシレータの倍音設計が不完全な場合でも、フィルタとエンベロープの設計によって楽器らしい音色を得られる可能性を示している。

5.3 カスタネットの打撃音合成の有効性

カスタネットの音源では、オシレータではなくノイズを音源とし、これを帯域通過フィルタと音量エンベロープに通す構成を採用した。評価の結果、合成音は立ち上がり直後に振幅が最大となり、約 20 ms で減衰する打撃音特有の波形を示した。これは、音程を持たない非調和的な打撃音と

いう、カスタネットの音響的特徴を再現できたことを示している。オシレータを用いる合成では明確な基本周波数を持つ音となり、打楽器の質感を再現することが難しいのに対し、ノイズを音源とし帯域通過フィルタで周波数帯を限定する手法は、打楽器音の合成に有効であったと考えられる。

5.4 改善案

以上の評価と考察を踏まえ、今後の改善点を述べる。第一に、主観評価が伸びなかった要因である音量の不足については、音源の出力レベルを見直すとともに、Processing 側でゲインを調整することでの改善が期待される。第二に、トロンボーンの音色については、実際の楽器の倍音構造をピーク検出によって正確に抽出し、波形テーブルに反映することで、より実物に近い音色が得られると考えられる。これらの改善により、客観評価に加えて主観評価においても基準を満たす音源の実現が期待される。

6 おわりに

本プロジェクトでは、幼児教育での活用を目的として、複数の楽器が同期して演奏し、あわせて演奏に同期して動作する人形による視覚的演出を行う分散型の自動演奏システムを開発した。このうち報告者は、トロンボーンおよびカスタネットの音源の設計と実装を担当した。

トロンボーンの音源では、オシレータによる波形にローパスフィルタ、ディレイ、および音量エンベロープを適用することで、管楽器に近い音色を合成した。カスタネットの音源では、オシレータではなくノイズを音源とし、これを帯域通過フィルタと音量エンベロープに通すことで、音程を持たない打撃音を再現した。

評価の結果、客観評価においては、担当したいずれの楽器も理論周波数との誤差が $\pm 1.0\%$ 以内という基準を満たし、実曲の演奏においても発音成功率 100%を達成した。一方、主観評価では、作成音の平均評価値が基準の 4.0 に達しなかった。これは、合成音の音量や、他の楽器との音量・音域の関係によって、担当した楽器の音が相対的に埋もれたことが要因と考えられる。また、トロンボーンの音色設計においては、生成 AI を用いた倍音の抽出に誤りがあったものの、フィルタおよびエンベロープによる処理によって、管楽器に近い音色を得ることができた。

今後の課題として、音源の音量や音量バランスの調整、および実際の楽器の倍音構造を正確に抽出して波形に反映することが挙げられる。これらの改善により、客観評価に加えて主観評価においても基準を満たす音源の実現が期待される。本システムは、音と視覚的演出を組み合わせることで、幼児のリズム教育に活用できる自動演奏システムとしての可能性を示すものである。

参考文献

- [1] 茂野仁美, 乳幼児保育におけるリズムへの同期の発達過程に関する文献研究. 大阪総合保育大学紀要, (14), 213-226, 2019.
- [2] 高橋むつみ, 篠田晴男, 幼児期におけるリズム知覚と協応動作の発達: 音楽活動を通した発

達支援の一助として、茨城大学教育学部紀要 (教育科学), (52), 249-262, 2003.

- [3] Lazzaro, J., & Wawryzynek, J. (2001). "A case for network musical performance." Proceedings of the 11th international workshop on Network and operating systems support for digital audio and video (NOSSDAV).
- [4] Rottondi, C., Chafe, C., Allocchio, C., & Sart, A. (2016). "An Overview on Networked Music Performance Technologies." IEEE Access, 4, 8823-8843.
- [5] D. Mills, J. Martin, J. Burbank and W. Kasch, "Network Time Protocol Version 4: Protocol and Algorithms Specification," RFC 5905, Jun. 2010.
- [6] IEEE, "IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems," IEEE Std 1588-2019 (Revision of IEEE Std 1588- 2008), pp. 1- 499, Nov. 2019.

A プログラムソース

A.1 報告者担当分

報告者が担当した、トロンボーンおよびカスタネットの音源のソースコードを示す。いずれも Processing で記述されており、クライアントからシリアル通信で受信した音符の情報をもとに音を合成して再生する。また、合成音の波形を描画し、基本周波数を測定して表示する機能を備える。

A.1.1 トロンボーンの音源

```
1 import processing.serial.*;
2 import ddf.minim.*;
3 import ddf.minim.ugens.*;
4
5 Serial myPort;
6 Minim minim;
7 AudioOutput out;
8 Waveform currentWaveform;
9 // --- 表示用変数 ---
10 String currentNote = "REST";
11 int currentBpm = -1; // 初期値は未設定にして外部から読み込む
12 String currentDurationName = "REST";
13
14 // 基準音符長 (デフォルト: 1.0 = 1拍単位)
15 float ToneLength = 1.0f;
16
17 float[] maxAmp = new float[29];
18 long lastNoteEndMillis = 0;
19 float currentToneLengthSec = 0.0f; // 現在鳴っている音の秒数
20 float currentToneLengthBeats = 0.0f; // 現在鳴っている音の拍数
21
```



```

22 // --- 周波数計測用 ---
23 float measuredFreq = 0.0f; // 自己相関法で計測した基本周波数(Hz)
24 float displayedFreq = 0.0f; // 表示用に平滑化した自己相関周波数(Hz)
25 float measuredZc = 0.0f; // ゼロ交差法で計測した基本周波数(Hz)
26 float displayedZc = 0.0f; // 表示用に平滑化したゼロ交差周波数(Hz)
27 float displayedPeak = 0.0f; // 表示用に平滑化したピーク振幅
28 float AMP_GATE = 0.01f; // この音量(RMS)未満は無音とみなし計測しない
29
30 // 目標周波数：C4(261.63Hz)を1オクターブ下げた値（本システムはfreq/2で発音）
31 float TARGET_FREQ = 261.63f / 2.0f; // = 130.815 Hz (C3相当)
32
33 // --- トロンボーンの特徴を反映したクラス ---
34 class HackInstrument implements Instrument
35 {
36     Oscil wave;
37     MoogFilter filter;
38     Delay delay;
39     ADSR ampEnv;
40     Line freqEnv;
41     float baseFreq;
42     float maxAmpValue;
43
44     HackInstrument(float frequency, float amplitude, Waveform wf)
45     {
46         baseFreq = frequency;
47         maxAmpValue = amplitude;
48         wave = new Oscil(frequency * 0.96f, 1.0, wf);
49
50         filter = new MoogFilter(1200.0f, 0.2f, MoogFilter.Type.LP);
51         delay = new Delay(0.2f, 0.3f, true, true);
52
53         // ADSRの引数：(最大音量, アタック秒, ディケイ秒, サステイン比率, リリース秒)
54         // 管楽器らしく 0.08秒かけて音が立ち上がり、吹いている間は 80% の音量をキープしま
55         // す
56         ampEnv = new ADSR(amplitude, 0.08f, 0.05f, 0.8f, 0.3f);
57
58         // 波形の出力を音量エンベロープ (ampEnv) にパッチする
59         wave.patch(filter).patch(delay).patch(ampEnv);
60
61         freqEnv = new Line();
62         freqEnv.patch(wave.frequency);
63     }
64
65     void noteOn(float duration)
66     {
67         ampEnv.noteOn(); // ADSRのトリガー開始
68         ampEnv.patch(out); // 出力ポートへ接続
69         freqEnv.activate(0.12f, baseFreq * 0.96f, baseFreq);

```

```

69     }
70
71     void noteOff()
72     {
73         ampEnv.noteOff(); // 音を消し始める（リリースタイム開始）
74         ampEnv.unpatchAfterRelease(out); // 音が完全に消え去った後に自動で接続を解除（ブ
            ツ切り防止）
75     }
76 }
77
78 void setup()
79 {
80     size(600, 300);
81     pixelDensity(1); // 高密度ディスプレイでの自動2倍描画を無効化（従来の見た目に戻す）
82     // 日本語が「□（豆腐）」にならないよう、日本語対応フォントを設定する
83     textFont(pickJapaneseFont(40));
84     textSize(24);
85     minim = new Minim(this);
86     out = minim.getLineOut();
87     out.setGain(2.0);
88     currentBpm = loadBpm();
89     out.setTempo(currentBpm);
90
91     // シリアルポートの初期化处理
92     // 利用可能なポートを表示し、Arduino (usbmodem / usbserial) を自動で探して接続する
93     printArray(Serial.list());
94     String portName = null;
95     for (int i = 0; i < Serial.list().length; i++) {
96         String s = Serial.list()[i];
97         if (s.indexOf("usbmodem") >= 0 || s.indexOf("usbserial") >= 0) {
98             portName = s;
99             break;
100         }
101     }
102     if (portName != null) {
103         myPort = new Serial(this, portName, 115200);
104         myPort.bufferUntil('\n');
105         println("Serial_connected:_ " + portName);
106     } else {
107         println("Arduinoのポートが見つかりません。上のリストから手動で指定してください。"
            );
108     }
109
110     for(int i=0; i<maxAmp.length; i++) maxAmp[i] = 0.15f;
111
112     setTromboneWave();
113     textSize(24);
114 }

```

```

115
116 void setTromboneWave() {
117     currentWaveform = WavetableGenerator.gen10(
118         4096,
119         new float[] {
120             1.0000f, 0.96f, 0.90f, 0.84f, 0.76f,
121             0.72f, 0.71f, 0.63f, 0.57f, 0.56f
122         }
123     );
124 }
125
126 // BPMを外部から取得する：環境変数 HCK_BPM または BPM、
127 // もしくはスケッチの data フォルダに置いた "bpm.txt" の1行目を読み込む。
128 int loadBpm() {
129     // 1) 環境変数
130     try {
131         String env = System.getenv("HCK_BPM");
132         if (env == null) env = System.getenv("BPM");
133         if (env != null) {
134             env = trim(env);
135             if (env.length() > 0) {
136                 try { return Integer.parseInt(env); } catch (Exception e) {}
137             }
138         }
139     } catch (Exception e) {
140         // ignore
141     }
142
143     // 2) data/bpm.txt
144     try {
145         String[] lines = loadStrings("bpm.txt");
146         if (lines != null && lines.length > 0) {
147             String s = trim(lines[0]);
148             if (s.length() > 0) {
149                 try { return Integer.parseInt(s); } catch (Exception e) {}
150             }
151         }
152     } catch (Exception e) {
153         // ファイルが無ければ無視
154     }
155
156     // 3) フォールバック
157     return 120;
158 }
159
160 void draw() {
161     background(0); // 画面を黒で塗りつぶしてリセット（オシロスコープ風）
162

```

```

163 // --- オシロスコープ風グリッド（目盛り）と軸の単位を描画 ---
164 // 縦方向の拡大率：振幅1.0が gain ピクセルに対応する
165 float gain = height * 1.5f;
166 // 画面横幅ぶんに対応する時間(ms)=バッファ長 / サンプリングレート
167 float totalMs = out.bufferSize() / out.sampleRate() * 1000.0f;
168 drawScopeGrid(gain, totalMs);
169
170 // 出力バッファ (out.mix) の波形を描画（正方向を上=オシロスコープと同じ向き）
171 stroke(0, 255, 120); // 波形の色（緑）
172 strokeWeight(1.5f);
173 noFill();
174 for (int i = 0; i < out.bufferSize() - 1; i++) {
175     float x1 = map(i, 0, out.bufferSize(), 0, width);
176     float x2 = map(i + 1, 0, out.bufferSize(), 0, width);
177     float y1 = height / 2 - out.mix.get(i) * gain;
178     float y2 = height / 2 - out.mix.get(i + 1) * gain;
179     line(x1, y1, x2, y2);
180 }
181
182 // --- 周波数を計測して画面に表示 ---
183 measuredFreq = detectFrequency(out.mix, out.sampleRate()); // 自己相関法
184 measuredZc = detectFreqZeroCross(out.mix, out.sampleRate()); // ゼロ交差法
185 float peak = peakAmplitude(out.mix); // ピーク振幅
186 if (measuredFreq > 0) {
187     // 急な変動を抑えるため指数移動平均で平滑化
188     displayedFreq = (displayedFreq <= 0) ? measuredFreq
189                     : displayedFreq * 0.8f + measuredFreq * 0.2f;
190     displayedZc = (displayedZc <= 0 || measuredZc <= 0) ? measuredZc
191                 : displayedZc * 0.8f + measuredZc * 0.2f;
192     displayedPeak = displayedPeak * 0.8f + peak * 0.2f;
193 } else {
194     displayedFreq = 0;
195     displayedZc = 0;
196     displayedPeak = 0; // 無音
197 }
198 drawFreqReadout(displayedFreq, displayedZc, displayedPeak);
199
200 // 保存リクエストがあれば、ヒント等を描く前のきれいな画面を保存する
201 if (saveRequested) {
202     String fn = "capture_" + timestamp() + ".png";
203     save(fn); // スケッチフォルダ直下に保存（波形+周波数のみ）
204     lastSavedName = fn;
205     savedFlashUntil = millis() + 1500;
206     println("画像を保存しました:_" + sketchPath(fn));
207     saveRequested = false;
208 }
209 }

```

```

210
211 // オシロスコープ風：薄い目盛りグリッド+原点を通る太い縦軸/横軸を描画する
212 void drawScopeGrid(float gain, float totalMs) {
213     pushStyle();
214     textSize(11);
215
216     float cy = height / 2.0f; // 振幅0 (横軸) のy
217     float ox = 2; // 時間0 (縦軸) のx (左端。太線が見えるよう少し内
        側)
218     float msPerDiv = 5.0f;
219     float ampPerDiv = 0.1f;
220     float ampMax = cy / gain; // 画面端に相当する振幅
221     int maxDiv = (int)(ampMax / ampPerDiv);
222
223     // ===== 1) 薄い目盛りグリッド (背景・控えめ) =====
224     strokeWeight(1);
225     stroke(38);
226     for (float t = 0; t <= totalMs + 0.001f; t += msPerDiv) { // 時間の縦線
227         float x = t / totalMs * width;
228         line(x, 0, x, height);
229     }
230     for (int k = -maxDiv; k <= maxDiv; k++) { // 振幅の横線
231         if (k == 0) continue;
232         float y = cy - k * ampPerDiv * gain;
233         line(0, y, width, y);
234     }
235
236     // ===== 2) 原点を通る太い軸 (はっきり) =====
237     stroke(0, 210, 255); // 水色で目立たせる (波形の緑と区別)
238     strokeWeight(2);
239     line(0, cy, width, cy); // 横軸 (時間 t: 振幅0のライン)
240     line(ox, 0, ox, height); // 縦軸 (振幅 A: 時間0のライン)
241
242     // 軸上の目盛りマーク (ティック)
243     stroke(0, 210, 255);
244     strokeWeight(1);
245     for (float t = msPerDiv; t < totalMs; t += msPerDiv) { // 横軸のティック
246         float x = t / totalMs * width;
247         line(x, cy - 4, x, cy + 4);
248     }
249     for (int k = -maxDiv; k <= maxDiv; k++) { // 縦軸のティック
250         if (k == 0) continue;
251         float y = cy - k * ampPerDiv * gain;
252         line(ox, y, ox + 8, y);
253     }
254
255     // ===== 3) 数値ラベル =====
256     noStroke();

```

```

257 textAlign(CENTER, TOP);
258 fill(190);
259 for (float t = msPerDiv; t < totalMs; t += msPerDiv) { // 時間[ms] (横軸の下)
260     float x = t / totalMs * width;
261     text(nf(t, 0, 0), x, cy + 6);
262 }
263 textAlign(RIGHT, CENTER);
264 for (int k = -maxDiv; k <= maxDiv; k++) { // 振幅 (右端)
265     if (k == 0) continue;
266     float a = k * ampPerDiv;
267     float y = cy - a * gain;
268     text((a > 0 ? "+" : "") + nf(a, 0, 1), width - 6, y);
269 }
270
271 // ===== 4) 軸タイトル・原点 =====
272 fill(0, 210, 255);
273 textAlign(RIGHT, TOP);
274 text("時間_t_[ms]_→", width - 6, cy + 22); // 横軸タイトル
275 textAlign(LEFT, TOP);
276 text("↑_振幅_A (正規化, _±
    1.0) ", ox + 6, 110); // 縦軸タイトル (左上の周波数パネルの下に置く)
277 textAlign(LEFT, TOP);
278 fill(190);
279 text("0", ox + 4, cy + 4); // 原点
280
281 // 目盛り幅の凡例 (左下)
282 textAlign(LEFT, BOTTOM);
283 text("1目盛り_横_" + nf(msPerDiv, 0, 0) + "_ms_/_縦_" + nf(ampPerDiv, 0, 1)
284     + " (フルスケール1.0) ", 6, height - 2);
285
286 popStyle();
287 }
288
289 // 画面左上に各計測値 (自己相関・ゼロ交差・ピーク振幅・目標誤差) を表示する
290 void drawFreqReadout(float fAuto, float fZc, float peak) {
291     pushStyle();
292     textAlign(LEFT, TOP);
293
294     // 文字が波形と重なっても読めるよう、背面に半透明の暗いパネルを敷く
295     noStroke();
296     fill(0, 170); // 黒・不透明度170/255 (波形がうっすら透ける程度)
297     rect(0, 0, 300, (fAuto > 0) ? 104 : 50);
298
299     if (fAuto > 0) {
300         // 自己相関法 (メイン)
301         fill(0, 255, 120);
302         textSize(26);
303         text(nf(fAuto, 0, 2) + "_Hz", 10, 5);

```

```

304
305     String name = nearestNoteName(fAuto);
306     float cents = centsOff(fAuto);
307     fill(200);
308     textSize(13);
309     text("自己相関__" + name + "__(" + sign(cents) + nf(cents, 0, 1) + "_cent)", 10,
        36);

310
311     // ゼロ交差法
312     if (fZc > 0) {
313         text("ゼロ交差__" + nf(fZc, 0, 2) + "_Hz", 10, 52);
314     } else {
315         text("ゼロ交差__--_Hz", 10, 52);
316     }
317
318     // ピーク振幅 (1.0超でクリップ)
319     text("ピーク振幅__" + nf(peak, 0, 3) + (peak >= 1.0f ? " (クリップ) " : " (クリッ
        プなし) "), 10, 68);

320
321     // 目標値に対する誤差 (自己相関法ベース)
322     float errHz = fAuto - TARGET_FREQ;
323     float errPct = errHz / TARGET_FREQ * 100.0f;
324     fill(errPct >= -1.0f && errPct <= 1.0f ? color(0, 255, 120) : color(255, 120,
        120));
325     text("目標__" + nf(TARGET_FREQ, 0, 2) + "Hz__誤差__"
326         + sign(errHz) + nf(errHz, 0, 2) + "Hz__/" + sign(errPct) + nf(errPct, 0, 2)
327         + "%",
328         10, 84);
329     } else {
330         fill(120);
331         textSize(26);
332         text("--_Hz", 10, 5);
333         textSize(13);
334         text(" (無音) ", 10, 36);
335     }
336     popStyle();
337 }
338
339 // 符号付き表示用 (+を明示)
340 String sign(float v) {
341     return v >= 0 ? "+" : "";
342 }
343
344 // 自己相関による基本周波数の推定 (低音域でも精度が出る)
345 float detectFrequency(AudioBuffer buf, float sampleRate) {
346     int n = buf.size();

```

```

346
347 // 1) 音量(RMS)を見て、無音なら計測しない
348 float rms = 0;
349 for (int i = 0; i < n; i++) {
350     float v = buf.get(i);
351     rms += v * v;
352 }
353 rms = sqrt(rms / n);
354 if (rms < AMP_GATE) return 0;
355
356 // 2) 探索する周期(ラグ)の範囲：約 50Hz ~ 1500Hz
357 int minLag = (int)(sampleRate / 1500.0f);
358 int maxLag = (int)(sampleRate / 50.0f);
359 if (maxLag > n - 1) maxLag = n - 1;
360 if (minLag < 2) minLag = 2;
361
362 // 3) 自己相関が最大になるラグを探す
363 float bestCorr = 0;
364 int bestLag = -1;
365 for (int lag = minLag; lag <= maxLag; lag++) {
366     float corr = 0;
367     for (int i = 0; i < n - lag; i++) {
368         corr += buf.get(i) * buf.get(i + lag);
369     }
370     if (corr > bestCorr) {
371         bestCorr = corr;
372         bestLag = lag;
373     }
374 }
375 if (bestLag <= 0) return 0;
376
377 // 4) 放物線補間でサブサンプル精度に補正
378 float lag = bestLag;
379 if (bestLag > minLag && bestLag < maxLag) {
380     float c0 = autocorrAt(buf, bestLag - 1);
381     float c1 = autocorrAt(buf, bestLag);
382     float c2 = autocorrAt(buf, bestLag + 1);
383     float denom = (c0 - 2 * c1 + c2);
384     if (abs(denom) > 1e-9) {
385         lag = bestLag + 0.5f * (c0 - c2) / denom;
386     }
387 }
388 return sampleRate / lag;
389 }
390
391 // 指定ラグでの自己相関値（補間用ヘルパー）
392 float autocorrAt(AudioBuffer buf, int lag) {
393     int n = buf.size();

```



```

394     float corr = 0;
395     for (int i = 0; i < n - lag; i++) {
396         corr += buf.get(i) * buf.get(i + lag);
397     }
398     return corr;
399 }
400
401 // ゼロ交差法による基本周波数の推定
402 // 倍音のさざ波を誤検出しないよう、ピーク振幅±の25%のヒステリシス（シュミットトリガ）
403 // 立ち上がりエッジのみを数え、ゼロ点を線形補間して最初と最後の交差から平均周期を求め
404 // る
405 float detectFreqZeroCross(AudioBuffer buf, float sampleRate) {
406     int n = buf.size();
407     float peak = peakAmplitude(buf);
408     if (peak < AMP_GATE) return 0;
409
410     float hi = peak * 0.25f; // 上側しきい値
411     float lo = -peak * 0.25f; // 下側しきい値
412     boolean wasLow = false; // 直近で下側しきい値を下回ったか
413     float firstCross = -1, lastCross = -1;
414     int cycles = 0;
415
416     for (int i = 1; i < n; i++) {
417         float v = buf.get(i);
418         if (v < lo) wasLow = true;
419         if (wasLow && v > hi) {
420             // 立ち上がり確定。直前サンプルとの間でゼロ点(=0)を線形補間
421             float v0 = buf.get(i - 1);
422             float pos = i;
423             if (v != v0) pos = (i - 1) + (0 - v0) / (v - v0);
424             if (firstCross < 0) {
425                 firstCross = pos;
426             } else {
427                 lastCross = pos;
428                 cycles++;
429             }
430             wasLow = false;
431         }
432     }
433
434     if (cycles < 1 || lastCross < 0) return 0;
435     float periodSamples = (lastCross - firstCross) / cycles;
436     if (periodSamples <= 0) return 0;
437     return sampleRate / periodSamples;
438 }
439
440 // バッファ内の最大振幅（絶対値）
441 float peakAmplitude(AudioBuffer buf) {

```

```

440     int n = buf.size();
441     float peak = 0;
442     for (int i = 0; i < n; i++) {
443         float a = abs(buf.get(i));
444         if (a > peak) peak = a;
445     }
446     return peak;
447 }
448
449 // 周波数に最も近い音名 (A4=440の12平均律)
450 String nearestNoteName(float f) {
451     String[] names = {"C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A", "A#", "B"};
452     int midi = round(69 + 12 * (log(f / 440.0f) / log(2)));
453     int octave = midi / 12 - 1;
454     return names[(midi % 12 + 12) % 12] + octave;
455 }
456
457 // 最寄り音からのズレ (セント, ±50が境目)
458 float centsOff(float f) {
459     float midi = 69 + 12 * (log(f / 440.0f) / log(2));
460     float nearest = round(midi);
461     return (midi - nearest) * 100.0f;
462 }
463
464 // --- 画面 (波形+周波数表示) を画像として保存 ---
465 // S キー: 現在の画面を PNG でスケッチフォルダに保存する
466 int savedFlashUntil = 0; // 「保存しました」表示を消す時刻(millis)
467 String lastSavedName = "";
468 boolean saveRequested = false; // S キーで立てる保存フラグ (実際の保存は draw 内で行う)
469
470 void keyPressed() {
471     if (key == 's' || key == 'S') {
472         saveRequested = true; // 描画タイミングに合わせて draw() 内で保存する
473     }
474 }
475
476 // 日本語を表示できるフォントを、この環境にあるものから自動で選ぶ
477 // (見つからなければ既定フォントにフォールバック)
478 PFont pickJapaneseFont(int sz) {
479     String[] avail = PFont.list();
480     // Mac/Win/Linux の代表的な日本語フォント名のキーワード (部分一致で探す)
481     String[] keys = {
482         "Hiragino", "YuGothic", "Yu_Gothic", "Noto_Sans_CJK", "Noto_Sans_JP",
483         "Noto_Serif_CJK", "MS_Gothic", "M S", "Meiryo", "メイリオ", "Osaka", "IPAGothic",
484         "TakaoGothic"
485     };
486 }

```

```

485     for (String k : keys) {
486         for (String a : avail) {
487             if (a.toLowerCase().indexOf(k.toLowerCase()) >= 0) {
488                 println("日本語フォントを使用:_" + a);
489                 return createFont(a, sz, true);
490             }
491         }
492     }
493     println("日本語フォントが見つかりませんでした。既定フォントを使用します。");
494     return createFont(avail.length > 0 ? avail[0] : "SansSerif", sz, true);
495 }
496
497 // ファイル名用のタイムスタンプ (例: 20260630_071530)
498 String timestamp() {
499     return year()
500         + nf(month(), 2) + nf(day(), 2) + "_"
501         + nf(hour(), 2) + nf(minute(), 2) + nf(second(), 2);
502 }
503
504 void serialEvent(Serial p) {
505     String data = p.readStringUntil('\n');
506     if (data != null) {
507         data = trim(data);
508         String[] values = split(data, ',');
509
510         // クライアントの送信形式: 「周波数, 鳴らすミリ秒, 強さ, BPM」 例: 262, 500, 100, 120
511         // "RAW_DATA:5" や "【BPM同期】 …" などのデバッグ行はカンマ区切りで4つに分かれな
512         // いので自動的に無視される
513         if (values.length >= 4) {
514             println(data); // 受信ノート記録 (正常発音数テスト用の受信ログ。不要なら削除)
515             try {
516                 // 1. 周波数(Hz)・ミリ秒・強さ・BPMを、それぞれ数値として読み込む
517                 float freq = float(values[0]) / 2.0f; // 周波数を半分にして1オクターブ下げる
518                 float durationMs = float(values[1]);
519                 // values[2] = velocity (強さ)。今回は音量に反映しないため未使用
520                 int bpmValue = int(float(values[3]));
521
522                 // BPMをクライアントに同期 (表示と拍数計算用)
523                 if (bpmValue > 0) {
524                     currentBpm = bpmValue;
525                     out.setTempo(currentBpm);
526                 }
527
528                 // 2. 周波数が0 (休符: REST) より大きいときだけ音を鳴らす
529                 if (freq > 0) {
530                     // 3. ミリ秒を1000で割って「秒」に変換する
531                     float durationSec = durationMs / 1000.0f;

```

```

532         currentNote = freq + "_Hz";
533         currentToneLengthSec = durationSec;
534         currentToneLengthBeats = durationMs / (60000.0f / currentBpm);
535         lastNoteEndMillis = millis() + (long)durationMs;
536
537         // 4. HackInstrumentに直接周波数 (freq) と秒数を渡す
538         out.playNote(0.0f, durationSec, new HackInstrument(freq, 0.15f,
539             currentWaveform));
540     }
541     catch (Exception e) {
542         // 例外が発生した場合は無視して続行
543     }
544 }
545 }
546 }

```

A.1.2 カスタネットの音源

```

1  import processing.serial.*;
2  import ddf.minim.*;
3  import ddf.minim.ugens.*;
4
5  Serial myPort;
6  Minim minim;
7  AudioOutput out;
8  Waveform currentWaveform;
9
10 // --- 表示用変数 ---
11 String currentNote = "REST";
12 int currentBpm = -1; // 初期値は未設定にして外部から読み込む
13 String currentDurationName = "REST";
14
15 // 基準音符長 (デフォルト: 1.0 = 1拍単位)
16 float ToneLength = 1.0f;
17
18 float[] maxAmp = new float[29];
19 long lastNoteEndMillis = 0;
20 float currentToneLengthSec = 0.0f; // 現在鳴っている音の秒数
21 float currentToneLengthBeats = 0.0f; // 現在鳴っている音の拍数
22
23 // --- カスタネットの特性を反映したクラス ---
24 // --- カスタネット: Arduino連動 & ユーザー微調整反映版 ---
25 class WoodPercussion implements Instrument
26 {
27     // 1. 表面の硬い音 (アタック成分)
28     Noise clickNoise;
29     ADSR clickEnv;

```

```

30 MoogFilter clickFilter;
31
32 // 2. 木の内部の響き（ボディ成分）
33 Noise bodyNoise;
34 ADSR bodyEnv;
35 MoogFilter bodyFilter;
36
37 // 3. 高域の乾いた成分（理想に近い参考版に合わせて追加）
38 Noise dryNoise;
39 ADSR dryEnv;
40 MoogFilter dryFilter;
41
42 WoodPercussion(float inputFreq, float amplitude)
43 {
44     // 参考版の音作り：周波数を木の共鳴帯域に寄せ、毎回わずかに揺らす
45     float bodyFreq = constrain(inputFreq, 1000.0f, 2600.0f);
46     bodyFreq += random(-90.0f, 90.0f);
47     float clickFreq = bodyFreq + random(2500.0f, 3600.0f);
48     float dryFreq = bodyFreq + random(4200.0f, 5200.0f);
49     float ampRand = amplitude * random(0.80f, 1.05f);
50
51     // ①【表面の硬い音】カチッという入りを残す
52     clickNoise = new Noise(ampRand * 1.15f, Noise.Tint.WHITE);
53     clickEnv = new ADSR(1.0f, 0.0005f, 0.007f, 0.0f, 0.003f);
54     clickFilter = new MoogFilter(clickFreq, 0.65f, MoogFilter.Type.BP);
55     clickNoise.patch(clickEnv).patch(clickFilter);
56
57     // ②【木の胴鳴り成分】「カッ」だけでなく「コッ」と響く感じを足す
58     bodyNoise = new Noise(ampRand * 0.40f, Noise.Tint.PINK);
59     bodyEnv = new ADSR(1.0f, 0.001f, 0.060f, 0.0f, 0.012f);
60     bodyFilter = new MoogFilter(bodyFreq, 0.85f, MoogFilter.Type.BP);
61     bodyNoise.patch(bodyEnv).patch(bodyFilter);
62
63     // ③【高域のパキッとした成分】硬さを残しつつ余韻を作る
64     dryNoise = new Noise(ampRand * 0.33f, Noise.Tint.WHITE);
65     dryEnv = new ADSR(1.0f, 0.0005f, 0.018f, 0.0f, 0.006f);
66     dryFilter = new MoogFilter(dryFreq, 0.55f, MoogFilter.Type.BP);
67     dryNoise.patch(dryEnv).patch(dryFilter);
68 }
69
70 void noteOn(float duration)
71 {
72     clickEnv.noteOn();
73     bodyEnv.noteOn();
74     dryEnv.noteOn();
75     clickFilter.patch(out);
76     bodyFilter.patch(out);
77     dryFilter.patch(out);

```

```

78     }
79
80     void noteOff()
81     {
82         clickEnv.noteOff();
83         bodyEnv.noteOff();
84         dryEnv.noteOff();
85         clickFilter.unpatch(out);
86         bodyFilter.unpatch(out);
87         dryFilter.unpatch(out);
88     }
89 }
90
91 void setup()
92 {
93     size(600, 300);
94     pixelDensity(1); // 高密度ディスプレイでの自動2倍描画を無効化（従来の見た目に戻す）
95     // 日本語が「□（豆腐）」に文字化けしないよう、日本語対応フォントを設定する
96     textFont(pickJapaneseFont(40));
97     textSize(24);
98     minim = new Minim(this);
99     out = minim.getLineOut();
100     out.setTempo(currentBpm);
101
102     // シリアルポートの初期化処理
103     // 利用可能なポートを表示し、Arduino (usbmodem / usbserial) を自動で探して接続する
104     printArray(Serial.list());
105     String portName = null;
106     for (int i = 0; i < Serial.list().length; i++) {
107         String s = Serial.list()[i];
108         if (s.indexOf("usbmodem") >= 0 || s.indexOf("usbserial") >= 0) {
109             portName = s;
110             break;
111         }
112     }
113     if (portName != null) {
114         myPort = new Serial(this, portName, 115200);
115         myPort.bufferUntil('\n');
116         println("Serial_connected:_" + portName);
117     } else {
118         println("Arduinoのポートが見つかりません。上のリストから手動で指定してください。"
119             );
120     }
121
122     for(int i=0; i<maxAmp.length; i++) maxAmp[i] = 0.15f;
123
124     setCastanetWave();
125     textSize(24);

```

```

125 }
126
127 // カスタネットの「カチッ」とした乾いたクラック音を再現するため、高次倍音を豊富に含ん
    // だ硬い波形を生成
128 void setCastanetWave() {
129     currentWaveform = WavetableGenerator.gen10(
130         4096,
131         new float[] { 0.1f, 0.3f, 0.5f, 0.8f, 1.0f, 0.9f, 0.8f, 0.7f, 0.6f, 0.5f }
132     );
133 }
134
135 void draw() {
136     background(0); // 画面を黒で塗りつぶしてリセット（オシロスコープ風）
137
138     // --- オシロスコープ風グリッド（目盛り）と軸の単位を描画 ---
139     float gain = height * 1.5f; // 振幅1.0が gain ピ
        // クセル
140     float totalMs = out.bufferSize() / out.sampleRate() * 1000.0f; // 画面横幅ぶんの時
        // 間(ms)
141     drawScopeGrid(gain, totalMs);
142
143     // 出力バッファ（out.mix）の波形を描画（正方向を上＝オシロスコープと同じ向き）
144     stroke(0, 255, 120); // 波形の色（緑）
145     strokeWeight(1.5f);
146     noFill();
147     for (int i = 0; i < out.bufferSize() - 1; i++) {
148         float x1 = map(i, 0, out.bufferSize(), 0, width);
149         float x2 = map(i + 1, 0, out.bufferSize(), 0, width);
150         float y1 = height / 2 - out.mix.get(i) * gain;
151         float y2 = height / 2 - out.mix.get(i + 1) * gain;
152         line(x1, y1, x2, y2);
153     }
154 }
155
156 // 日本語を表示できるフォントを、この環境にあるものから自動で選ぶ
157 PFont pickJapaneseFont(int sz) {
158     String[] avail = PFont.list();
159     String[] keys = {
160         "Hiragino", "YuGothic", "Yu_Gothic", "Noto_Sans_CJK", "Noto_Sans_JP",
161         "Noto_Serif_CJK", "MS_Gothic", "Meiryo", "メイリオ", "Osaka", "IPAGothic", "
        TakaoGothic"
162     };
163     for (String k : keys) {
164         for (String a : avail) {
165             if (a.toLowerCase().indexOf(k.toLowerCase()) >= 0) {
166                 println("日本語フォントを使用:_" + a);
167                 return createFont(a, sz, true);
168             }

```

```

169     }
170 }
171 println("日本語フォントが見つかりませんでした。既定フォントを使用します。");
172 return createFont(avail.length > 0 ? avail[0] : "SansSerif", sz, true);
173 }
174
175 // オシロスコープ風：薄い目盛りグリッド+原点を通る太い縦軸/横軸を描画する
176 void drawScopeGrid(float gain, float totalMs) {
177     pushStyle();
178     textSize(11);
179
180     float cy = height / 2.0f;        // 振幅0（横軸）のy
181     float ox = 2;                    // 時間0（縦軸）のx（左端。太線が見えるよう少し内
                                     // 側）
182     float msPerDiv = 5.0f;
183     float ampPerDiv = 0.1f;
184     float ampMax = cy / gain;        // 画面端に相当する振幅
185     int maxDiv = (int)(ampMax / ampPerDiv);
186
187     // ===== 1) 薄い目盛りグリッド（背景・控えめ） =====
188     strokeWeight(1);
189     stroke(38);
190     for (float t = 0; t <= totalMs + 0.001f; t += msPerDiv) {    // 時間の縦線
191         float x = t / totalMs * width;
192         line(x, 0, x, height);
193     }
194     for (int k = -maxDiv; k <= maxDiv; k++) {                    // 振幅の横線
195         if (k == 0) continue;
196         float y = cy - k * ampPerDiv * gain;
197         line(0, y, width, y);
198     }
199
200     // ===== 2) 原点を通る太い軸（はっきり） =====
201     stroke(0, 210, 255);    // 水色で目立たせる（波形の緑と区別）
202     strokeWeight(2);
203     line(0, cy, width, cy);    // 横軸（時間 t：振幅0のライン）
204     line(ox, 0, ox, height);    // 縦軸（振幅 A：時間0のライン）
205
206     // 軸上の目盛りマーク（ティック）
207     stroke(0, 210, 255);
208     strokeWeight(1);
209     for (float t = msPerDiv; t < totalMs; t += msPerDiv) {    // 横軸のティック
210         float x = t / totalMs * width;
211         line(x, cy - 4, x, cy + 4);
212     }
213     for (int k = -maxDiv; k <= maxDiv; k++) {                    // 縦軸のティック
214         if (k == 0) continue;
215         float y = cy - k * ampPerDiv * gain;

```



```

216     line(ox, y, ox + 8, y);
217 }
218
219 // ===== 3) 数値ラベル =====
220 noStroke();
221 textAlign(CENTER, TOP);
222 fill(190);
223 for (float t = msPerDiv; t < totalMs; t += msPerDiv) { // 時間[ms] (横軸の下)
224     float x = t / totalMs * width;
225     text(nf(t, 0, 0), x, cy + 6);
226 }
227 textAlign(RIGHT, CENTER);
228 for (int k = -maxDiv; k <= maxDiv; k++) { // 振幅 (右端)
229     if (k == 0) continue;
230     float a = k * ampPerDiv;
231     float y = cy - a * gain;
232     text((a > 0 ? "+" : "") + nf(a, 0, 1), width - 6, y);
233 }
234
235 // ===== 4) 軸タイトル・原点 =====
236 fill(0, 210, 255);
237 textAlign(RIGHT, TOP);
238 text("時間_t_[ms]_→", width - 6, cy + 22); // 横軸タイトル
239 textAlign(LEFT, TOP);
240 text("↑_振幅_A (正規化, ±1.0) ", ox + 6, 8); // 縦軸タイトル
241 fill(190);
242 text("0", ox + 4, cy + 4); // 原点
243
244 // 目盛り幅の凡例 (左下)
245 textAlign(LEFT, BOTTOM);
246 text("1目盛り_横_ " + nf(msPerDiv, 0, 0) + "_ms_/_縦_" + nf(ampPerDiv, 0, 1)
247     + " (フルスケール1.0) ", 6, height - 2);
248
249 popStyle();
250 }
251
252 void keyPressed() {
253     // 画像保存 (S) : 現在の波形画面を PNG で保存
254     if (key == 's' || key == 'S') {
255         String ts = year() + nf(month(), 2) + nf(day(), 2) + "_"
256             + nf(hour(), 2) + nf(minute(), 2) + nf(second(), 2);
257         save("castanet_capture_" + ts + ".png");
258         println("画像を保存しました :_castanet_capture_" + ts + ".png");
259     }
260 }
261
262 void serialEvent(Serial p) {
263     String data = p.readStringUntil('\n');

```

```

264 if (data != null) {
265     data = trim(data);
266     String[] values = split(data, ',');
267
268     // クライアントの送信形式：「周波数,鳴らすミリ秒,強さ,BPM」 例：262,500,100,120
269     // "RAW_DATA:5" や "【BPM同期】 …" などのデバッグ行はカンマ区切りで4つに分かれな
        いので自動的に無視される
270     if (values.length >= 4) {
271         println(data); // 受信ノート記録（正常発音数テスト用の受信ログ。不要なら削除）
272         try {
273             // 1. 周波数(Hz)・ミリ秒・強さ・BPMをそれぞれ数値としてパース
274             // カスタネットらしい抜けの良い高域を引き出すため、送られてきた周波数をそのま
                ま使用
275             float freq = float(values[0]);
276             float durationMs = float(values[1]);
277             // values[2] = velocity（強さ）。今回は音量に反映しないため未使用
278             int bpmValue = int(float(values[3]));
279
280             // BPMをクライアントに同期（表示と拍数計算用）
281             if (bpmValue > 0) {
282                 currentBpm = bpmValue;
283                 out.setTempo(currentBpm);
284             }
285
286             // 2. 周波数が0（休符）より大きいときだけ打音を生成
287             if (freq > 0) {
288                 float durationSec = durationMs / 1000.0f;
289
290                 currentNote = freq + "_Hz";
291                 currentToneLengthSec = durationSec;
292                 currentToneLengthBeats = durationMs / (60000.0f / currentBpm);
293                 lastNoteEndMillis = millis() + (long)durationMs;
294
295                 // 3. カスタネット特性を持たせたインストゥルメントオブジェクトを生成して再
                    生
296                 out.playNote(0.0f, 0.8f, new WoodPercussion(freq, 1.0f));
297             }
298         }
299         catch (Exception e) {
300             // データ破損などの例外発生時は安全に処理をスキップ
301         }
302     }
303 }
304 }

```

A.2 担当箇所以外

システムの根幹となる，サーバおよびクライアントのソースコードを示す．これらは主に他のメンバーが実装したものである．

A.2.1 サーバ (server.ino)

```
1 #include <WiFiS3.h>
2 #include <WiFiUdp.h>
3 #include "server_function.h"
4 #include "Arduino_LED_Matrix.h"
5
6 // 【ネットワーク設定】
7 char ssid[] = "hackathon003-WPA2";
8 char pass[] = "hackathon003";
9 IPAddress BcAddress(255, 255, 255, 255); // 全員に一斉送信（ブロードキャスト）
10
11 uint16_t Port = 3000;
12 WiFiUDP udp;
13
14 // 【システム設定用変数】
15 ArduinoLEDMatrix matrix;
16
17 // --- 小節管理用 ---
18 uint32_t LastBarTime = 0;
19 uint16_t Interval = 2000;
20 uint8_t BarCount = 39;
21
22 // --- BPM管理用 ---
23 uint8_t BPM = 120;
24 uint32_t LastPressTime = 0;
25 bool Flag = false;
26
27
28 void setup() {
29     Serial.begin(115200);
30     matrix.begin();
31
32     // スイッチピンの設定
33     pinMode(2, INPUT);
34     pinMode(3, INPUT);
35
36     // 初期Intervalの計算
37     Interval = (60000 / BPM) * 4;
38
39     // Wi-Fi接続処理
40     Serial.print("サーバー：Wi-Fiに接続中:_)");
```

```

41 Serial.println(ssid);
42 WiFi.begin(ssid, pass);
43 while (WiFi.status() != WL_CONNECTED) {
44     delay(500);
45     Serial.print(".");
46 }
47 Serial.println("");
48
49 // IPアドレス取得待機
50 while (WiFi.localIP() == IPAddress(0, 0, 0, 0)) {
51     Serial.println("ルーターからIPアドレスを取得中...");
52     delay(500);
53 }
54
55 Serial.println("接続完了!");
56 Serial.print("サーバー自身のIPアドレス:");
57 Serial.println(WiFi.localIP());
58
59 udp.begin(Port);
60 }
61
62
63 void loop() {
64     Bar_control(&LastBarTime, Interval, &BarCount, BCaddress, Port, udp);
65     BPM_control(&BPM, &Interval, &LastPressTime, &Flag, BCaddress, Port, udp);
66 }

```

サーバのメインプログラムである。setup() で Wi-Fi への接続とスイッチピンの初期化を行い、loop() で小節番号の送信 (Bar_control) とテンポの変更・送信 (BPM_control) を繰り返し実行する。

A.2.2 サーバの関数ヘッダ (server_function.h)

```

1 #ifndef SERVER_FUNCTION_H
2 #define SERVER_FUNCTION_H
3 #include <Arduino.h>
4 #include <WiFiUdp.h>
5 #include "Arduino_LED_Matrix.h"
6
7 void Bar_control(uint32_t *LastBarTime, uint16_t Interval, uint8_t *BarCount,
8     IPAddress BCaddress, uint16_t Port, WiFiUDP &udp);
9
10
11 void BPM_control(uint8_t *BPM, uint16_t *Interval, uint32_t *LastPressTime, bool *
12     Flag, IPAddress BCaddress, uint16_t Port, WiFiUDP &udp);
13
14 #endif

```

サーバの主要な関数のプロトタイプ宣言である。

A.2.3 サーバの関数実装 (server_function.cpp)

```
1 #include "server_function.h"
2
3 // =====
4 // スイッチ押下検出ピン
5 // =====
6 #define PIN_BPM_UP 2 // BPMアップ
7 #define PIN_BPM_DOWN 3 // BPMダウン
8 // =====
9 // デバウンスタイム
10 // =====
11 #define DEBOUNCE_MS 30
12
13 void Bar_control(uint32_t *LastBarTime, uint16_t Interval, uint8_t *BarCount,
14                 IPAddress BCaddress, uint16_t Port, WiFiUDP &udp) {
15     uint32_t temp = millis();
16
17     // Interval (間隔) 以上時間が経過したかチェック
18     if (temp - *LastBarTime >= Interval) {
19         *LastBarTime = temp;
20
21         // 小節番号のカウント (上限39)
22         if (*BarCount >= 39) {
23             *BarCount = 0;
24         } else {
25             *BarCount += 1;
26         }
27
28         // UDPブロードキャスト送信
29         udp.beginPacket(BCaddress, Port);
30         udp.write(*BarCount);
31         udp.endPacket();
32
33         // デバッグ用出力
34         Serial.print("送信した小節番号:");
35         Serial.println(*BarCount);
36     }
37 }
38 // -----
39 // チャタリング制御用関数
40 // -----
41 static bool detectPress(uint8_t pin, int *lastReading, int *stableState, uint32_t *
    lastChangeTime) {
    int reading = digitalRead(pin);
```

```

42
43 // 読み値が変化したら、変化時刻を更新
44 if (reading != *lastReading) {
45     *lastChangeTime = millis();
46     *lastReading = reading;
47 }
48
49 // 一定時間読み値が安定していたら確定状態を更新する
50 if (millis() - *lastChangeTime > DEBOUNCE_MS) {
51     if (reading != *stableState) {
52         *stableState = reading;
53         // 確定状態が HIGH に変わった瞬間 = 押された瞬間
54         if (*stableState == HIGH) {
55             return true;
56         }
57     }
58 }
59 return false;
60 }
61
62
63 void BPM_control(uint8_t *BPM, uint16_t *Interval, uint32_t *LastPressTime, bool *
    Flag, IPAddress Bcaddress, uint16_t Port, WiFiUDP &udp) {

64
65 // --- 各ボタンのデバウンス状態（関数をまたいで保持するため static） ---
66 static int      upLastReading   = LOW;
67 static int      upStableState   = LOW;
68 static uint32_t upLastChange    = 0;
69
70 static int      downLastReading = LOW;
71 static int      downStableState = LOW;
72 static uint32_t downLastChange  = 0;
73
74 // === BPMアップ (D2) ===
75 if (detectPress(PIN_BPM_UP, &upLastReading, &upStableState, &upLastChange)) {
76     if (*BPM <= 250) { // 上限255を超えない範囲で (+5してもオーバーフ
        ローしない)
77         *BPM += 5;
78         *Flag = true;
79         *LastPressTime = millis(); // 最後に押した時刻
80         Serial.print("BPM_UP!_現在のBPM:_"); Serial.println(*BPM);
81     }
82 }
83
84 // === BPMダウン (D3) ===
85 if (detectPress(PIN_BPM_DOWN, &downLastReading, &downStableState, &downLastChange))
    {

```

```

86     if (*BPM >= 45) { // 下限40を下回らない範囲で (-5してもアンダーフ
        ローしない)
87         *BPM -= 5;
88         *Flag = true;
89         *LastPressTime = millis();
90         Serial.print("BPM_DOWN!_現在のBPM:_"); Serial.println(*BPM);
91     }
92 }
93
94 // === BPM変更後の送信処理 ===
95 // 最後の押下から1秒以上 何も押されなければ、まとめて1回だけ送信する
96 if (*Flag) {
97     if (millis() - *LastPressTime > 1000) {
98         udp.beginPacket(BCaddress, Port);
99         udp.write(*BPM);
100        udp.endPacket();
101
102        *Flag = false;
103
104        // 新しいBPMから小節の時間間隔(Interval)を再計算
105        // 1分=60000ミリ秒。4/4拍子なので4倍
106        *Interval = (60000 / *BPM) * 4;
107
108        Serial.print("【送信完了】 BPM値を全楽器に送信しました。新Interval:_");
109        Serial.println(*Interval);
110    }
111 }
112 }

```

サーバの主要な処理を実装したファイルである。Bar_control 関数は、一定間隔ごとに小節番号を 0 から 39 の範囲で循環させながら UDP で送信する。detectPress 関数は、スイッチの読み値が 30ms 安定して初めて押下を確定することで、チャタリングによる誤検出を防ぐ。BPM_control 関数は、スイッチ押下に応じてテンポを 5 ずつ増減させ、最後の押下から 1 秒経過後に新しいテンポを送信する。

A.2.4 クライアント (client.ino)

楽器クライアントのメインプログラムである。setup() で Wi-Fi への接続と譜面データの初期化を行い、loop() で受信データの判別 (Parse_data)、テンポの更新 (BPM_update)、演奏処理 (Performance)、および人形の駆動 (robot.update) を繰り返し実行する。各クライアントは、輪唱を実現するための固有のオフセット値 (Offset) を持ち、受信した小節番号にこれを加えることで自身の演奏開始位置をずらす。これにより、同一の譜面を複数のクライアントが時間差で演奏し、輪唱として成立させている。

```

1 #include <WiFiS3.h>
2 #include <WiFiUdp.h>

```

```

3 #include "client_function.h"
4 #include "EntertainmentRobot.h"
5
6
7 //Offset変数とsetupScore関数を楽器に合わせて下さい。その他の、client_function.cpp,
  client_function.hはいじらなくても大丈夫です。
8
9
10 // --- ネットワーク設定 ---
11 char ssid[] = "hackathon003-WPA2";
12 char pass[] = "hackathon003";
13 uint16_t Port = 3000; // サーバーと同じポート番号
14
15 WiFiUDP Udp;
16
17 EntertainmentRobot robot;
18
19 // --- クライアントサイドのグローバル変数 ---
20 char Offset = -2; // 輪唱に必要なオフセット値（楽器ごとに設定、2番手だと-2）
21 uint8_t Data = 255; // 受信データ
22 uint8_t CurrentBPM = 120; // 現在のBPM（初期値はサーバーに合わせて120とする）
23 uint8_t CurrentBar = 255; // 現在演奏中の小節番号
24 uint8_t NoteIndex = 255; // 現在の小節の中で、何番目の音符を鳴らしているかのインデックス
25 bool Flag = false; // 受信データが小節番号かBPMかを判定するフラグ
26 float ToneLength = 0.0; // 基準音符（4分音符）の音の長さ（ミリ秒）
27 uint32_t StartTime = 0; // 前回の音符を鳴らした時間（millis）
28 uint16_t Interval = 0; // 実際に演奏する音の長さ（ミリ秒）
29
30 Pfm Score[25]; // 楽譜（40小節分の配列）
31
32
33 // --- 周波数定義 ---
34 #define NOTE_C4 262 // ド
35 #define NOTE_D4 294 // レ
36 #define NOTE_E4 330 // ミ
37 #define NOTE_F4 349 // ファ
38 #define NOTE_G4 392 // ソ
39 #define NOTE_A4 440 // ラ
40 #define REST 0 // 休符
41
42 #define NOTE_C5 523 // 高いド
43 #define NOTE_D5 587 // 高いレ
44 #define NOTE_E5 659 // 高いミ
45 #define NOTE_F5 698 // 高いファ
46 #define NOTE_G5 784 // 高いソ
47 #define NOTE_A5 880 // 高いラ

```



```

48
49 void setupScore() {
50     // 第0小節～第7小節 (1周目)
51     Score[0] = {{ {NOTE_C4, 1.0, 92}, {NOTE_D4, 1.0, 96}, {NOTE_E4, 1.0, 100}, {NOTE_F4,
52         1.0, 104} }, 4};
53     Score[1] = {{ {NOTE_E4, 1.0, 100}, {NOTE_D4, 1.0, 96}, {NOTE_C4, 1.0, 90}, {REST,
54         1.0, 0} }, 4};
55     Score[2] = {{ {NOTE_E4, 1.0, 96}, {NOTE_F4, 1.0, 100}, {NOTE_G4, 1.0, 106}, {
56         NOTE_A4, 1.0, 112} }, 4};
57     Score[3] = {{ {NOTE_G4, 1.0, 106}, {NOTE_F4, 1.0, 100}, {NOTE_E4, 1.0, 94}, {REST,
58         1.0, 0} }, 4};
59     Score[4] = {{ {NOTE_C4, 1.0, 104}, {REST, 1.0, 0}, {NOTE_C4, 1.0, 104}, {REST, 1.0,
60         0} }, 4};
61     Score[5] = {{ {NOTE_C4, 1.0, 98}, {REST, 1.0, 0}, {NOTE_C4, 1.0, 106}, {REST, 1.0,
62         0} }, 4};
63     Score[6] = {{ {NOTE_C4, 0.5, 88}, {NOTE_C4, 0.5, 92}, {NOTE_D4, 0.5, 96}, {NOTE_D4,
64         0.5, 100}, {NOTE_E4, 0.5, 104}, {NOTE_E4, 0.5, 108}, {NOTE_F4, 0.5, 112}, {
65         NOTE_F4, 0.5, 116} }, 8};
66     Score[7] = {{ {NOTE_E4, 1.0, 108}, {NOTE_D4, 1.0, 100}, {NOTE_C4, 1.0, 92}, {REST,
67         1.0, 0} }, 4};
68
69     // 第8小節 (休符)
70     Score[8] = {{ {REST, 4.0, 0} }, 1};
71     Score[9] = {{ {REST, 4.0, 0} }, 1};
72     Score[10] = {{ {REST, 4.0, 0} }, 1};
73     Score[11] = {{ {REST, 4.0, 0} }, 1};
74     Score[12] = {{ {REST, 4.0, 0} }, 1};
75
76     // 第9小節～第16小節 (2周目：1オクターブ上)
77     Score[13] = {{ {NOTE_C5, 1.0, 92}, {NOTE_D5, 1.0, 96}, {NOTE_E5, 1.0, 100}, {
78         NOTE_F5, 1.0, 104} }, 4};
79     Score[14] = {{ {NOTE_E5, 1.0, 100}, {NOTE_D5, 1.0, 96}, {NOTE_C5, 1.0, 90}, {REST,
80         1.0, 0} }, 4};
81     Score[15] = {{ {NOTE_E5, 1.0, 96}, {NOTE_F5, 1.0, 100}, {NOTE_G5, 1.0, 106}, {
82         NOTE_A5, 1.0, 112} }, 4};
83     Score[16] = {{ {NOTE_G5, 1.0, 106}, {NOTE_F5, 1.0, 100}, {NOTE_E5, 1.0, 94}, {REST,
84         1.0, 0} }, 4};
85     Score[17] = {{ {NOTE_C5, 1.0, 104}, {REST, 1.0, 0}, {NOTE_C5, 1.0, 104}, {REST,
86         1.0, 0} }, 4};
87     Score[18] = {{ {NOTE_C5, 1.0, 98}, {REST, 1.0, 0}, {NOTE_C5, 1.0, 106}, {REST, 1.0,
88         0} }, 4};
89     Score[19] = {{ {NOTE_C5, 0.5, 88}, {NOTE_C5, 0.5, 92}, {NOTE_D5, 0.5, 96}, {NOTE_D5,
90         0.5, 100}, {NOTE_E5, 0.5, 104}, {NOTE_E5, 0.5, 108}, {NOTE_F5, 0.5, 112}, {
91         NOTE_F5, 0.5, 116} }, 8};
92     Score[20] = {{ {NOTE_E5, 1.0, 108}, {NOTE_D5, 1.0, 100}, {NOTE_C5, 1.0, 92}, {REST,
93         1.0, 0} }, 4};

```

```

76
77 // 第17小節～第18小節（休符）
78 Score[21] = {{ {REST, 4.0, 0} }, 1};
79 Score[22] = {{ {REST, 4.0, 0} }, 1};
80 Score[23] = {{ {REST, 4.0, 0} }, 1};
81 Score[24] = {{ {REST, 4.0, 0} }, 1};
82
83
84 }
85
86
87 void setup() {
88   Serial.begin(115200);
89
90   Serial.println("Wi-Fiに接続中...");
91   WiFi.begin(ssid, pass);
92   while (WiFi.status() != WL_CONNECTED) {
93     delay(500);
94     Serial.print(".");
95   }
96   Serial.println("\nWi-Fi接続完了!");
97
98   // IPアドレス取得待機
99   while (WiFi.localIP() == IPAddress(0, 0, 0, 0)) {
100     Serial.println("ルーターからIPアドレスを取得中...");
101     delay(500);
102   }
103   Serial.print("クライアントのIPアドレス:");
104   Serial.println(WiFi.localIP());
105
106   Udp.begin(Port);
107
108   setupScore();
109   ToneLength = 60000.0 / CurrentBPM;
110   Serial.println("クライアント：譜面データの読み込みが完了しました。");
111   robot.begin(ssid, pass, Port);
112 }
113
114 void loop() {
115   Flag = Parse_data(&Data, Offset, Udp);
116
117   // 2. データがBPMだった場合の同期処理
118   BPM_update(&CurrentBPM, Flag, Data, &ToneLength);
119
120   // 3. 演奏位置制御とPCへのデータ送信（今回追加）
121   Performance(Data, &CurrentBar, &NoteIndex, &StartTime, &Interval, ToneLength,
122               CurrentBPM, Score);
123   robot.update();

```

A.2.5 クライアントの関数ヘッダ (client_function.h)

譜面データの構造体 (NoteData, Pfm) と、クライアントの主要な関数のプロトタイプを定義したファイルである。NoteData は 1 つの音符の周波数・長さ・強さを、Pfm は 1 小節分の音符列を表す。

```

1  #ifndef CLIENT_FUNCTION_H
2  #define CLIENT_FUNCTION_H
3  #include <Arduino.h>
4  #include <WiFiUdp.h>
5
6  // --- 譜面データの構造体定義 ---
7  // 1つの音符の情報を表す構造体
8  struct NoteData {
9      int pitch;          // 音の高さ (周波数 Hz)
10     float length;       // 音の長さ (基準となる4分音符を 1.0 とした相対値)
11     uint8_t velocity;   // 音の強さ (0~127)
12 };
13
14 // 1つの小節 (Pfm) の情報を表す構造体
15 struct Pfm {
16     NoteData notes[8]; // 1小節に最大8つの音符が入ると仮定
17     int noteCount;     // この小節に実際に含まれる音符の数
18 };
19
20 bool Parse_data(uint8_t *Data, char Offset, WiFiUDP &Udp); // 受信データがBPM値なら
    true (フラグON), 小節番号ならfalse (フラグOFF) を返す関数
21
22 void BPM_update(uint8_t *CurrentBPM, bool Flag, uint8_t Data, float *ToneLength);
23
24 void Performance(uint8_t Data, uint8_t *CurrentBar, uint8_t *NoteIndex, uint32_t *
    StartTime, uint16_t *Interval, float ToneLength, uint8_t CurrentBPM, Pfm *Score);
25 #endif

```

A.2.6 クライアントの関数実装 (client_function.cpp)

クライアントの主要な処理を実装したファイルである。Parse_data 関数は、受信した 1 バイトのデータを値の大小によってテンポと小節番号に判別し、小節番号にはオフセットを加える。BPM_update 関数は、受信したテンポから基準となる 4 分音符の長さを算出する。Performance 関数は、受信した小節番号に対応する音符を譜面データから順に読み出し、周波数・長さ・強さ・テンポをシリアル通信で Processing へ送信する。

```

1  #include "client_function.h"
2

```

```

3  bool Parse_data(uint8_t *Data, char Offset, WiFiUDP &Udp) {
4      int packetSize = Udp.parsePacket();
5
6      // 1. パケットを受信したかどうかの判定
7      if (packetSize) {
8          uint8_t received = Udp.read();
9          Udp.flush();
10
11         // === 【生存確認用デバッグ】受信した生の数値をそのままProcessingへ送る ===
12         Serial.print("RAW_DATA:");
13         Serial.println(received);
14         // =====
15
16         // 2. 受信データが40以上であればBPM値と判定（フラグONを返す）
17         if (received >= 25) {
18             *Data = received;
19             return true; // フラグON
20         }
21         // 3. 40未満であれば小節番号と判定（オフセットを適用し、フラグOFFを返す）
22         else {
23             // サーバーの小節番号とオフセットを足す
24             int myBar = received + Offset;
25
26             // マイナスになる場合（まだ自分の出番ではない待機期間）
27             if (myBar < 0) {
28                 *Data = 255; // 待機状態を維持するための無効な値を入れる
29             } else {
30                 *Data = myBar; // 自分の小節番号として適用
31             }
32             return false; // フラグOFF
33         }
34     }
35 }
36 return false;
37 }
38
39 void BPM_update(uint8_t *CurrentBPM, bool Flag, uint8_t Data, float *ToneLength) {
40     // フラグがOFF（小節番号）の場合は何もせず終了
41     if (!Flag) {
42         return;
43     }
44     // フラグがON（BPM値）の場合の更新処理
45     else {
46         *CurrentBPM = Data;
47
48         // 基準となる4分音符の長さをミリ秒単位で計算（60000 / BPM）
49         *ToneLength = 60000.0 / (*CurrentBPM);
50     }

```

```

51 // 確認用のシリアル出力
52 Serial.print("【BPM同期】新しいBPM:");
53 Serial.print(*CurrentBPM);
54 Serial.print("_->4分音符の基準長さ(ms):");
55 Serial.println(*ToneLength);
56 }
57 }
58
59 void Performance(uint8_t Data, uint8_t *CurrentBar, uint8_t *NoteIndex, uint32_t *
    StartTime, uint16_t *Interval, float ToneLength, uint8_t CurrentBPM, Pfm *Score)
    {

60
61 // 1. 新しい小節番号を受信した時のリセット処理
62 // (Dataが40未満で、かつ現在演奏中の小節と異なる場合)
63 if (Data < 24 && Data != *CurrentBar) {
64     *CurrentBar = Data; // 現在の小節番号を更新
65     *NoteIndex = 0; // 読み込む音符を最初(0番目)に戻す
66     *StartTime = millis(); // 演奏開始タイマーをリセット
67     *Interval = 0; // 即座に最初の音が鳴るようにIntervalを0にする
68 }
69
70 // フリーズ防止
71 if (*CurrentBar >= 40) {
72     return;
73 }
74
75 // 2. 音を鳴らすタイミング(時間が経過したか)の判定
76 if (millis() - *StartTime >= *Interval) {
77
78 // その小節にある音符の数(noteCount)に達するまで処理を行う
79 if (*NoteIndex < Score[*CurrentBar].noteCount) {
80
81     *StartTime = millis(); // 次の音の基準時間を現在時刻に更新
82
83 // 譜面配列から「音の高さ(pitch)」と「音の長さの割合(length)」を取得
84 int pitch = Score[*CurrentBar].notes[*NoteIndex].pitch;
85 float length = Score[*CurrentBar].notes[*NoteIndex].length;
86 uint8_t velocity = Score[*CurrentBar].notes[*NoteIndex].velocity;
87
88 // 次の音までの間隔(Interval)を計算(ミリ秒)
89 *Interval = ToneLength * length;
90
91 // PC (Processing) ヘシリアル通信で送信
92 // 形式: 「周波数,鳴らすミリ秒,強さ,BPM」 (例: 262,500,100,120)
93 Serial.print(pitch);
94 Serial.print(",");
95 Serial.print(*Interval);

```

```

96     Serial.print(",");
97     Serial.print(velocity);
98     Serial.print(",");
99     Serial.println(CurrentBPM);
100
101     *NoteIndex = *NoteIndex + 1; // 次の音符へ進む
102 }
103 }
104 }

```

A.2.7 人形制御クラスのヘッダ (EntertainmentRobot.h)

演奏に同期して人形のサーボモータを駆動する EntertainmentRobot クラスを定義したファイルである。楽器の種類（ピアノ・トロンボーン・バイオリン・カスタネット）に応じたサーボの角度や動作パターン，および楽譜に対応した動作テーブルを保持する。人形側にも輪唱のためのオフセット（PART_BAR_OFFSET）が設定されている。

```

1  #ifndef ENTERTAINMENT_ROBOT_H
2  #define ENTERTAINMENT_ROBOT_H
3
4  #include <Arduino.h>
5  #include <WiFiS3.h>
6  #include <WiFiUdp.h>
7  #include <Servo.h>
8
9  // =====
10 // 楽器選択定義
11 // =====
12 #define PIANO      1
13 #define TROMBONE   2
14 #define VIOLIN     3
15 #define CASTANET   4
16
17 // 使う人形に合わせてここを変更
18 // #define INSTRUMENT PIANO
19 #define INSTRUMENT TROMBONE
20 // #define INSTRUMENT VIOLIN
21 // #define INSTRUMENT CASTANET
22
23 // =====
24 // 楽譜データ駆動のための構造体・列挙型
25 // =====
26 enum ScoreEventType {
27     SCORE_END   = 0,
28     SCORE_PLAY  = 1,
29     SCORE_REST   = 2
30 };

```

```

31
32 enum NoteLengthUnit {
33     LENGTH_SIXTEENTH = 1,
34     LENGTH_EIGHTH    = 2,
35     LENGTH_QUARTER    = 4,
36     LENGTH_HALF       = 8,
37     LENGTH_WHOLE      = 16
38 };
39
40 struct ServoScoreEvent {
41     ScoreEventType type;
42     uint8_t lengthUnits;
43 };
44
45 class EntertainmentRobot {
46 public:
47     EntertainmentRobot();
48     void begin(const char* ssid, const char* pass, uint16_t port);
49     void update();
50
51 private:
52     // 通信関連
53     WiFiUDP _udp;
54     uint16_t _localPort;
55
56     // サーボオブジェクト
57     Servo _rightArmServo;
58     Servo _leftArmServo;
59     Servo _extraServo;
60
61     // ピン配置定数
62     static const int RIGHT_ARM_PIN = 9;
63     static const int LEFT_ARM_PIN  = 11; // 新コードに合わせて10番に修正
64     static const int EXTRA_PIN    = 5;
65
66     // サーボ角度定数
67     static const int ARM_REST_ANGLE = 90;
68     static const int PIANO_RIGHT_PLAY_ANGLE = 70;
69     static const int PIANO_LEFT_PLAY_ANGLE  = 110;
70
71     static const int TROMBONE_FORWARD_ANGLE = 65;
72     static const int TROMBONE_BACK_ANGLE    = 115;
73
74     static const int VIOLIN_BOW_LEFT_ANGLE  = 65;
75     static const int VIOLIN_BOW_RIGHT_ANGLE = 115;
76
77     static const int CASTANET_PLAY_ANGLE    = 70;
78

```

```

79  static const int HEAD_CENTER_ANGLE      = 90;
80  static const int HEAD_LEFT_ANGLE       = 90; // 70度に修正
81  static const int HEAD_RIGHT_ANGLE      = 180; // 110度に修正
82
83  static const int MOUTH_CLOSE_ANGLE     = 90; // 90度に修正
84  static const int MOUTH_OPEN_ANGLE      = 180; // 60度に修正
85
86  // 全休符・オフセット設定
87  static const uint8_t SERVER_BAR_COUNT = 25;
88  static const int PART_BAR_OFFSET = -2; // 必要に応じて変更してください
89  static const bool WHOLE_REST_BAR_TABLE[SERVER_BAR_COUNT];
90
91  // 楽譜テーブルの定数
92  static const uint8_t MAX_EVENTS_PER_BAR = 8;
93  static const uint8_t BAR_LENGTH_UNITS = 16;
94  static const uint8_t QUARTER_LENGTH_UNITS = 4;
95  static const ServoScoreEvent SCORE_TABLE[SERVER_BAR_COUNT][MAX_EVENTS_PER_BAR];
96
97  // 状態管理変数
98  uint8_t _currentBPM;
99  uint8_t _currentBar;
100 unsigned long _beatInterval;
101
102 // 楽譜イベント駆動用のタイマー変数
103 unsigned long _nextEventTime;
104 uint8_t _scoreEventIndex;
105 uint8_t _usedUnitsInBar;
106
107 bool _barActive;
108 bool _armDirection;
109 bool _motionMutedByWholeRest; // 全休符ミュートフラグ
110 bool _scoreRestActive;       // 小節内の部分休符フラグ
111 bool _pianoUseRightArm;      // ピアノの左右腕交互フラグ
112
113 // サーボパルス（タイマー）管理
114 bool _rightArmReturnActive;
115 bool _leftArmReturnActive;
116 bool _extraReturnActive;
117 unsigned long _rightArmReturnTime;
118 unsigned long _leftArmReturnTime;
119 unsigned long _extraReturnTime;
120
121 // BPM変更エフェクト管理
122 bool _emotionActive;
123 unsigned long _emotionEndTime;
124 unsigned long _lastEmotionMoveTime;
125 bool _emotionDirection;
126 static const int EMOTION_BEATS = 4;

```



```

127
128 // 内部メソッド
129 void setupServos();
130 void receiveUdpData();
131 void updateBPM(uint8_t newBPM);
132 void calculateBeatInterval();
133 unsigned long noteLengthToMs(uint8_t lengthUnits);
134 unsigned long calculatePulseTime(unsigned long eventDuration);
135 uint8_t toScoreBar(uint8_t serverBarNumber);
136 bool isWholeRestBar(uint8_t scoreBarNumber);
137 void startBarMotion(uint8_t barNumber);
138 void updateNormalMotion();
139 void triggerNextScoreEvent();
140 void triggerScoreMotion(unsigned long eventDuration);
141 void finishBarMotion();
142 void startEmotion();
143 void updateEmotionMotion();
144
145 void triggerRightArmPulse(int angle, unsigned long pulseTime);
146 void triggerLeftArmPulse(int angle, unsigned long pulseTime);
147 void triggerExtraPulse(int angle, unsigned long pulseTime);
148 void updateServoReturn();
149 void stopAllMotionForWholeRest();
150 void stopMainMotionForRest();
151 void cancelServoReturns();
152 void resetMainServos();
153 void resetEmotionServo();
154 bool timeReached(unsigned long targetTime);
155 };
156
157 #endif

```

A.2.8 人形制御クラスの実装 (EntertainmentRobot.cpp)

EntertainmentRobot クラスの実装である。受信した小節番号とテンポに基づいてサーボを駆動し、演奏に同期した人形の動作を行う。また、テンポ変更時には、通常の演奏動作とは別の演出動作を行う。

```

1 #include "EntertainmentRobot.h"
2
3 // =====
4 // 全休符テーブルの定義 (false: 演奏, true: 全休符)
5 // =====
6 const bool EntertainmentRobot::WHOLE_REST_BAR_TABLE[SERVER_BAR_COUNT] = {
7     false, false, false, false, false, false, false, false, // 0~7小節 (1周目)
8     true, true, true, true, true, // 8~12小節 (休符)
9     false, false, false, false, false, false, false, false, // 13~20小節 (2周目)
10    true, true, true, true // 21~24小節 (休符)

```

```

11 };
12
13 // =====
14 // 楽譜テーブルの定義 (25小節目)
15 // =====
16 #define SCORE_FILL          { SCORE_END, 0 }
17
18 #define BAR_QUARTER_4        { \
19     {SCORE_PLAY, LENGTH_QUARTER}, {SCORE_PLAY, LENGTH_QUARTER}, \
20     {SCORE_PLAY, LENGTH_QUARTER}, {SCORE_PLAY, LENGTH_QUARTER}, \
21     SCORE_FILL, SCORE_FILL, SCORE_FILL, SCORE_FILL \
22 }
23
24 #define BAR_QUARTER_3_REST_1 { \
25     {SCORE_PLAY, LENGTH_QUARTER}, {SCORE_PLAY, LENGTH_QUARTER}, \
26     {SCORE_PLAY, LENGTH_QUARTER}, {SCORE_REST, LENGTH_QUARTER}, \
27     SCORE_FILL, SCORE_FILL, SCORE_FILL, SCORE_FILL \
28 }
29
30 #define BAR_EIGHTH_8         { \
31     {SCORE_PLAY, LENGTH_EIGHTH}, {SCORE_PLAY, LENGTH_EIGHTH}, \
32     {SCORE_PLAY, LENGTH_EIGHTH}, {SCORE_PLAY, LENGTH_EIGHTH}, \
33     {SCORE_PLAY, LENGTH_EIGHTH}, {SCORE_PLAY, LENGTH_EIGHTH}, \
34     {SCORE_PLAY, LENGTH_EIGHTH}, {SCORE_PLAY, LENGTH_EIGHTH} \
35 }
36
37 #define BAR_WHOLE_REST        { \
38     {SCORE_REST, LENGTH_WHOLE}, \
39     SCORE_FILL, SCORE_FILL, SCORE_FILL, SCORE_FILL, SCORE_FILL, SCORE_FILL \
40 }
41
42 #define BAR_NOTE_REST_NOTE_REST { \
43     {SCORE_PLAY, LENGTH_QUARTER}, {SCORE_REST, LENGTH_QUARTER}, \
44     {SCORE_PLAY, LENGTH_QUARTER}, {SCORE_REST, LENGTH_QUARTER}, \
45     SCORE_FILL, SCORE_FILL, SCORE_FILL, SCORE_FILL \
46 }
47
48 const ServoScoreEvent EntertainmentRobot::SCORE_TABLE[SERVER_BAR_COUNT][
49     MAX_EVENTS_PER_BAR] = {
50     BAR_QUARTER_4,           // 0小節目
51     BAR_QUARTER_3_REST_1,    // 1小節目
52     BAR_QUARTER_4,           // 2小節目
53     BAR_QUARTER_3_REST_1,    // 3小節目
54     BAR_NOTE_REST_NOTE_REST, // 4小節目
55     BAR_NOTE_REST_NOTE_REST, // 5小節目
56     BAR_EIGHTH_8,           // 6小節目
57     BAR_QUARTER_3_REST_1,    // 7小節目

```

```

57
58   BAR_WHOLE_REST,          // 8小節目
59   BAR_WHOLE_REST,          // 9小節目
60   BAR_WHOLE_REST,          // 10小節目
61   BAR_WHOLE_REST,          // 11小節目
62   BAR_WHOLE_REST,          // 12小節目
63
64   BAR_QUARTER_4,           // 13小節目
65   BAR_QUARTER_3_REST_1,    // 14小節目
66   BAR_QUARTER_4,           // 15小節目
67   BAR_QUARTER_3_REST_1,    // 16小節目
68   BAR_NOTE_REST_NOTE_REST, // 17小節目
69   BAR_NOTE_REST_NOTE_REST, // 18小節目
70   BAR_EIGHTH_8,            // 19小節目
71   BAR_QUARTER_3_REST_1,    // 20小節目
72
73   BAR_WHOLE_REST,          // 21小節目
74   BAR_WHOLE_REST,          // 22小節目
75   BAR_WHOLE_REST,          // 23小節目
76   BAR_WHOLE_REST           // 24小節目
77 };
78
79 #undef SCORE_FILL
80 #undef BAR_QUARTER_4
81 #undef BAR_QUARTER_3_REST_1
82 #undef BAR_EIGHTH_8
83 #undef BAR_WHOLE_REST
84 #undef BAR_NOTE_REST_NOTE_REST
85
86
87 EntertainmentRobot::EntertainmentRobot() :
88     _currentBPM(120), _currentBar(0), _nextEventTime(0),
89     _scoreEventIndex(0), _usedUnitsInBar(0),
90     _barActive(false), _armDirection(false), _motionMutedByWholeRest(false),
91     _scoreRestActive(true), _pianoUseRightArm(true),
92     _rightArmReturnActive(false), _leftArmReturnActive(false), _extraReturnActive(
93         false),
94     _rightArmReturnTime(0), _leftArmReturnTime(0), _extraReturnTime(0),
95     _emotionActive(false), _lastEmotionMoveTime(0), _emotionDirection(false)
96 {
97     calculateBeatInterval();
98 }
99
100 void EntertainmentRobot::begin(const char* ssid, const char* pass, uint16_t port) {
101     _localPort = port;
102
103     setupServos();
104     calculateBeatInterval();

```

```

104
105     Serial.println("Connecting_to_Wi-Fi...");
106     while (WiFi.begin(ssid, pass) != WL_CONNECTED) {
107         Serial.print(".");
108         delay(1000);
109     }
110     Serial.println("\nWi-Fi_Connected!");
111
112     _udp.begin(_localPort);
113
114     Serial.println("Entertainment_client_started");
115     Serial.print("IP:");
116     Serial.println(WiFi.localIP());
117 }
118
119 void EntertainmentRobot::update() {
120     receiveUdpData();
121     updateNormalMotion();
122     updateEmotionMotion();
123     updateServoReturn();
124 }
125
126 void EntertainmentRobot::setupServos() {
127     #if INSTRUMENT == PIANO
128         _rightArmServo.attach(RIGHT_ARM_PIN);
129         _leftArmServo.attach(LEFT_ARM_PIN);
130         _rightArmServo.write(ARM_REST_ANGLE);
131         _leftArmServo.write(ARM_REST_ANGLE);
132
133     #elif INSTRUMENT == TROMBONE
134         _rightArmServo.attach(RIGHT_ARM_PIN);
135         _rightArmServo.write(ARM_REST_ANGLE);
136
137     #elif INSTRUMENT == VIOLIN
138         _rightArmServo.attach(RIGHT_ARM_PIN);
139         _extraServo.attach(EXTRA_PIN);
140         _rightArmServo.write(ARM_REST_ANGLE);
141         _extraServo.write(HEAD_CENTER_ANGLE);
142
143     #elif INSTRUMENT == CASTANET
144         _rightArmServo.attach(RIGHT_ARM_PIN);
145         _extraServo.attach(EXTRA_PIN);
146         _rightArmServo.write(ARM_REST_ANGLE);
147         _extraServo.write(MOUTH_CLOSE_ANGLE);
148     #endif
149 }
150
151 void EntertainmentRobot::receiveUdpData() {

```

```

152     int packetSize = _udp.parsePacket();
153
154     if (packetSize > 0) {
155         int data = _udp.read();
156
157         while (_udp.available()) {
158             _udp.read();
159         }
160
161         if (data < 0) {
162             return;
163         }
164
165         // 40以上（または25以上）をBPMとして判定するための境界
166         if (data >= 40) {
167             updateBPM((uint8_t)data);
168         } else {
169             startBarMotion((uint8_t)data);
170         }
171     }
172 }
173
174 void EntertainmentRobot::updateBPM(uint8_t newBPM) {
175     if (newBPM == _currentBPM) {
176         return;
177     }
178
179     _currentBPM = newBPM;
180     calculateBeatInterval();
181
182     Serial.print("BPM_updated:_");
183     Serial.println(_currentBPM);
184
185     startEmotion();
186 }
187
188 void EntertainmentRobot::calculateBeatInterval() {
189     _beatInterval = 60000UL / _currentBPM;
190 }
191
192 unsigned long EntertainmentRobot::noteLengthToMs(uint8_t lengthUnits) {
193     if (lengthUnits == 0) return 0;
194     return (_beatInterval * (unsigned long)lengthUnits) / QUARTER_LENGTH_UNITS;
195 }
196
197 unsigned long EntertainmentRobot::calculatePulseTime(unsigned long eventDuration) {
198     if (eventDuration == 0) return 0;
199     unsigned long pulseTime = (eventDuration * 7UL) / 10UL;

```

```

200
201     if (pulseTime < 40UL) { pulseTime = 40UL; }
202     if (eventDuration > 30UL && pulseTime > eventDuration - 20UL) {
203         pulseTime = eventDuration - 20UL;
204     }
205     if (pulseTime > eventDuration) { pulseTime = eventDuration; }
206
207     return pulseTime;
208 }
209
210 uint8_t EntertainmentRobot::toScoreBar(uint8_t serverBarNumber) {
211     int scoreBar = (int)serverBarNumber + PART_BAR_OFFSET;
212     scoreBar %= SERVER_BAR_COUNT;
213
214     if (scoreBar < 0) {
215         scoreBar += SERVER_BAR_COUNT;
216     }
217
218     return (uint8_t)scoreBar;
219 }
220
221 bool EntertainmentRobot::isWholeRestBar(uint8_t scoreBarNumber) {
222     if (scoreBarNumber >= SERVER_BAR_COUNT) {
223         return false;
224     }
225     return WHOLE_REST_BAR_TABLE[scoreBarNumber];
226 }
227
228 void EntertainmentRobot::startBarMotion(uint8_t barNumber) {
229     uint8_t scoreBar = toScoreBar(barNumber);
230     _currentBar = scoreBar;
231
232     Serial.print("Server_bar:_");
233     Serial.print(barNumber);
234     Serial.print("_->_Score_bar:_");
235     Serial.println(_currentBar);
236
237     if (isWholeRestBar(_currentBar)) {
238         _motionMutedByWholeRest = true;
239         _scoreRestActive = true;
240         stopAllMotionForWholeRest();
241         return;
242     }
243
244     _motionMutedByWholeRest = false;
245     _scoreRestActive = false;
246
247     _nextEventTime = millis();

```

```

248     _scoreEventIndex = 0;
249     _usedUnitsInBar = 0;
250     _barActive = true;
251 }
252
253 void EntertainmentRobot::updateNormalMotion() {
254     if (_motionMutedByWholeRest || !_barActive) {
255         return;
256     }
257
258     while (_barActive && timeReached(_nextEventTime)) {
259         triggerNextScoreEvent();
260     }
261 }
262
263 void EntertainmentRobot::triggerNextScoreEvent() {
264     if (_scoreEventIndex >= MAX_EVENTS_PER_BAR || _usedUnitsInBar >= BAR_LENGTH_UNITS
265         ) {
266         finishBarMotion();
267         return;
268     }
269
270     ServoScoreEvent event = SCORE_TABLE[_currentBar][_scoreEventIndex];
271     _scoreEventIndex++;
272
273     if (event.type == SCORE_END || event.lengthUnits == 0) {
274         finishBarMotion();
275         return;
276     }
277
278     uint8_t lengthUnits = event.lengthUnits;
279     if (_usedUnitsInBar + lengthUnits > BAR_LENGTH_UNITS) {
280         lengthUnits = BAR_LENGTH_UNITS - _usedUnitsInBar;
281     }
282
283     unsigned long eventDuration = noteLengthToMs(lengthUnits);
284
285     if (event.type == SCORE_REST) {
286         _scoreRestActive = true;
287         stopMainMotionForRest();
288     } else if (event.type == SCORE_PLAY) {
289         _scoreRestActive = false;
290         triggerScoreMotion(eventDuration);
291     }
292
293     _usedUnitsInBar += lengthUnits;
294     _nextEventTime += eventDuration;
295 }

```

```

295
296 void EntertainmentRobot::triggerScoreMotion(unsigned long eventDuration) {
297     unsigned long pulseTime = calculatePulseTime(eventDuration);
298
299     #if INSTRUMENT == PIANO
300         if (_pianoUseRightArm) {
301             triggerRightArmPulse(PIANO_RIGHT_PLAY_ANGLE, pulseTime);
302         } else {
303             triggerLeftArmPulse(PIANO_LEFT_PLAY_ANGLE, pulseTime);
304         }
305         _pianoUseRightArm = !_pianoUseRightArm;
306
307     #elif INSTRUMENT == TROMBONE
308         _armDirection = !_armDirection;
309         if (_armDirection) {
310             _rightArmServo.write(TROMBONE_FORWARD_ANGLE);
311         } else {
312             _rightArmServo.write(TROMBONE_BACK_ANGLE);
313         }
314
315     #elif INSTRUMENT == VIOLIN
316         _armDirection = !_armDirection;
317         if (_armDirection) {
318             _rightArmServo.write(VIOLIN_BOW_LEFT_ANGLE);
319         } else {
320             _rightArmServo.write(VIOLIN_BOW_RIGHT_ANGLE);
321         }
322
323     #elif INSTRUMENT == CASTANET
324         triggerRightArmPulse(CASTANET_PLAY_ANGLE, pulseTime);
325     #endif
326 }
327
328 void EntertainmentRobot::finishBarMotion() {
329     _barActive = false;
330     _scoreRestActive = true;
331     stopMainMotionForRest();
332 }
333
334 void EntertainmentRobot::startEmotion() {
335     if (_motionMutedByWholeRest || _scoreRestActive) {
336         return;
337     }
338
339     #if INSTRUMENT == VIOLIN || INSTRUMENT == CASTANET
340         _emotionActive = true;
341         _emotionEndTime = millis() + _beatInterval * EMOTION_BEATS;
342         _lastEmotionMoveTime = 0;

```



```

343     _emotionDirection = false;
344 #endif
345 }
346
347 void EntertainmentRobot::updateEmotionMotion() {
348     if (_motionMutedByWholeRest || _scoreRestActive) {
349         resetEmotionServo();
350         _emotionActive = false;
351         return;
352     }
353
354     if (!_emotionActive) {
355         return;
356     }
357
358     unsigned long now = millis();
359     if (timeReached(_emotionEndTime)) {
360         resetEmotionServo();
361         _emotionActive = false;
362         return;
363     }
364
365     unsigned long emotionInterval = _beatInterval / 2;
366     if (emotionInterval < 100) {
367         emotionInterval = 100;
368     }
369
370     if (now - _lastEmotionMoveTime >= emotionInterval) {
371         _lastEmotionMoveTime = now;
372         _emotionDirection = !_emotionDirection;
373
374         #if INSTRUMENT == VIOLIN
375             if (_emotionDirection) {
376                 _extraServo.write(HEAD_LEFT_ANGLE);
377             } else {
378                 _extraServo.write(HEAD_RIGHT_ANGLE);
379             }
380         #elif INSTRUMENT == CASTANET
381             if (_emotionDirection) {
382                 _extraServo.write(MOUTH_OPEN_ANGLE);
383             } else {
384                 _extraServo.write(MOUTH_CLOSE_ANGLE);
385             }
386         #endif
387     }
388 }
389
390 void EntertainmentRobot::triggerRightArmPulse(int angle, unsigned long pulseTime) {

```

```

391     _rightArmServo.write(angle);
392     _rightArmReturnActive = true;
393     _rightArmReturnTime = millis() + pulseTime;
394 }
395
396 void EntertainmentRobot::triggerLeftArmPulse(int angle, unsigned long pulseTime) {
397     _leftArmServo.write(angle);
398     _leftArmReturnActive = true;
399     _leftArmReturnTime = millis() + pulseTime;
400 }
401
402 void EntertainmentRobot::triggerExtraPulse(int angle, unsigned long pulseTime) {
403     _extraServo.write(angle);
404     _extraReturnActive = true;
405     _extraReturnTime = millis() + pulseTime;
406 }
407
408 void EntertainmentRobot::updateServoReturn() {
409     unsigned long now = millis();
410
411     if (_rightArmReturnActive && timeReached(_rightArmReturnTime)) {
412         _rightArmServo.write(ARM_REST_ANGLE);
413         _rightArmReturnActive = false;
414     }
415
416     if (_leftArmReturnActive && timeReached(_leftArmReturnTime)) {
417         _leftArmServo.write(ARM_REST_ANGLE);
418         _leftArmReturnActive = false;
419     }
420
421     if (_extraReturnActive && timeReached(_extraReturnTime)) {
422         resetEmotionServo();
423         _extraReturnActive = false;
424     }
425 }
426
427 void EntertainmentRobot::stopAllMotionForWholeRest() {
428     _barActive = false;
429     _scoreEventIndex = 0;
430     _usedUnitsInBar = 0;
431     _emotionActive = false;
432     cancelServoReturns();
433     resetMainServos();
434     resetEmotionServo();
435 }
436
437 void EntertainmentRobot::stopMainMotionForRest() {
438     _rightArmReturnActive = false;

```

```

439     _leftArmReturnActive = false;
440     resetMainServos();
441 }
442
443 void EntertainmentRobot::cancelServoReturns() {
444     _rightArmReturnActive = false;
445     _leftArmReturnActive = false;
446     _extraReturnActive = false;
447 }
448
449 void EntertainmentRobot::resetMainServos() {
450     #if INSTRUMENT == PIANO
451         _rightArmServo.write(ARM_REST_ANGLE);
452         _leftArmServo.write(ARM_REST_ANGLE);
453     #elif INSTRUMENT == TROMBONE
454         _rightArmServo.write(ARM_REST_ANGLE);
455     #elif INSTRUMENT == VIOLIN
456         _rightArmServo.write(ARM_REST_ANGLE);
457     #elif INSTRUMENT == CASTANET
458         _rightArmServo.write(ARM_REST_ANGLE);
459     #endif
460 }
461
462 void EntertainmentRobot::resetEmotionServo() {
463     #if INSTRUMENT == VIOLIN
464         _extraServo.write(HEAD_CENTER_ANGLE);
465     #elif INSTRUMENT == CASTANET
466         _extraServo.write(MOUTH_CLOSE_ANGLE);
467     #endif
468 }
469
470 bool EntertainmentRobot::timeReached(unsigned long targetTime) {
471     return (long)(millis() - targetTime) >= 0;
472 }

```

A.2.9 カスタネットクライアント (castanets_client.ino)

カスタネット用のクライアントプログラムである。共通のクライアントプログラム (client.ino) を、音程を持たないカスタネット用に変更したものである。音階を並べたメロディの代わりに、固定周波数 (1800 Hz) の打撃と休符を組み合わせた打撃パターンを譜面データとして保持する。

```

1 #include <WiFiS3.h>
2 #include <WiFiUdp.h>
3 #include "castanets_client_function.h"
4
5 // Offset変数とsetupScore関数を楽器に合わせて下さい。その他の、client_function.cpp,

```

client_function.hはいじらなくても大丈夫です。

```
6
7 // --- ネットワーク設定 ---
8 char ssid[] = "hackathon003-WPA2"; //
9 char pass[] = "hackathon003"; //
10 uint16_t Port = 3000; // サーバーと同じポート番号
11
12 WiFiUDP Udp;
13
14 // --- クライアントサイドのグローバル変数 ---
15 char Offset = 0; // 輪唱に必要なオフセット値（楽器ごとに設定，2番手だと-2）
16 uint8_t Data = 255; // 受信データ
17 uint8_t CurrentBPM = 120; // 現在のBPM（初期値はサーバーに合わせて120とする）
18 uint8_t CurrentBar = 255; // 現在演奏中の小節番号
19 uint8_t NoteIndex = 255; // 現在の小節の中で、何番目の音符を鳴らしているかのインデックス
20 bool Flag = false; // 受信データが小節番号かBPMかを判定するフラグ
21 float ToneLength = 0.0; // 基準音符（4分音符）の音の長さ（ミリ秒）
22 uint32_t StartTime = 0; // 前回の音符を鳴らした時間（millis）
23 uint16_t Interval = 0; // 実際に演奏する音の長さ（ミリ秒）
24
25 Pfm Score[40]; // 楽譜（40小節分の配列）
26
27 // --- 周波数定義 ---
28 constexpr int kFreqValue = 1800; // 音階なしのため固定の周波数
29 constexpr int kRest = 0;
30
31 void setupScore()
32 {
33     // 第0小節～第7小節（1周目）
34     Score[0] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
35                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
36     Score[1] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
37                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
38     Score[2] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
39                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
40     Score[3] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
41                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
42     Score[4] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
43                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
44     Score[5] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
45                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
46     Score[6] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
47                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
48     Score[7] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
49                 {kRest, 0.8f}, {kRest, 1.0f}}, 8};
```

```

42
43 // 第8小節（休符）
44 Score[8] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
               {kRest, 0.8f}, {kRest, 1.0f}}, 8};

45
46 // 第9小節～第16小節（2周目：1オクターブ上）
47 Score[9] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
               {kRest, 0.8f}, {kRest, 1.0f}}, 8};
48 Score[10] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
49 Score[11] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
50 Score[12] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
51 Score[13] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
52 Score[14] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
53 Score[15] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
54 Score[16] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};

55
56 // 第17小節～第18小節（休符）
57 Score[17] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
58 Score[18] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};

59
60 // 第19小節～第26小節（3周目：通常）
61 Score[19] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
62 Score[20] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
63 Score[21] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
64 Score[22] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
65 Score[23] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
66 Score[24] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
67 Score[25] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
                {kRest, 0.8f}, {kRest, 1.0f}}, 8};
68 Score[26] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},

```

```

        {kRest, 0.8f}, {kRest, 1.0f}}, 8);

69
70 // 第27小節（休符）
71 Score[27] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);

72
73 // 第28小節～第35小節（4周目：1オクターブ上）
74 Score[28] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);
75 Score[29] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);
76 Score[30] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);
77 Score[31] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);
78 Score[32] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);
79 Score[33] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);
80 Score[34] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);
81 Score[35] = {{{kFreqValue, 0.2f}, {kRest, 0.8f}, {kRest, 1.0f}, {kFreqValue, 0.2f},
        {kRest, 0.8f}, {kRest, 1.0f}}, 8);

82
83 // 第36小節～第39小節（終了・全休符）
84 for (int i = 36; i < 40; i++)
85 {
86     Score[i] = {{{kRest, 4.0f}}, 1);
87 }
88 }
89
90
91 void setup()
92 {
93     Serial.begin(115200);
94
95     Serial.println("Wi-Fiに接続中...");
96     WiFi.begin(ssid, pass);
97     while (WiFi.status() != WL_CONNECTED)
98     {
99         delay(500);
100         Serial.print(".");
101     }
102     Serial.println("\nWi-Fi接続完了!");
103

```

```

104 // IPアドレス取得待機
105 while (WiFi.localIP() == IPAddress(0, 0, 0, 0))
106 {
107     Serial.println("ルーターからIPアドレスを取得中...");
108     delay(500);
109 }
110 Serial.print("クライアントのIPアドレス:");
111 Serial.println(WiFi.localIP());
112
113 Udp.begin(Port);
114
115 setupScore();
116 ToneLength = 60000.0 / CurrentBPM;
117 Serial.println("クライアント：譜面データの読み込みが完了しました。");
118 }
119
120 void loop()
121 {
122     // 1. 受信パケットの仕分けと小節番号のオフセット適用
123     Flag = Parse_data(&Data, Offset, Udp);
124
125     // 2. データがBPMだった場合の同期処理
126     BPM_update(&CurrentBPM, Flag, Data, &ToneLength);
127
128     // 3. 演奏位置制御とPCへのデータ送信（今回追加）
129     Performance(Data, &CurrentBar, &NoteIndex, &StartTime, &Interval, ToneLength, Score
130 );
131 }

```

A.2.10 カスタネットクライアントの関数ヘッダ (castanets_client_function.h)

カスタネット用の譜面データ構造体と関数のプロトタイプを定義したファイルである。カスタネットは音の強さをを用いないため、共通版の NoteData 構造体から強さ (velocity) を除いている。

```

1 #ifndef CLIENT_FUNCTION_H
2 #define CLIENT_FUNCTION_H
3 #include <Arduino.h>
4 #include <WiFiUdp.h>
5
6 // --- 譜面データの構造体定義 ---
7 // 1つの音符の情報を表す構造体
8 struct NoteData {
9     int pitch; // 音の高さ（周波数 Hz）
10    float length; // 音の長さ（基準となる4分音符を 1.0 とした相対値）
11 };
12
13 // 1つの小節 (Pfm) の情報を表す構造体

```

```

14 struct Pfm {
15     NoteData notes[8]; // 1小節に最大8つの音符が入ると仮定
16     int noteCount;      // この小節に実際に含まれる音符の数
17 };
18
19 bool Parse_data(uint8_t *Data, char Offset, WiFiUDP &Udp); // 受信データがBPM値なら
    true (フラグON), 小節番号ならfalse (フラグOFF) を返す関数
20
21 void BPM_update(uint8_t *CurrentBPM, bool Flag, uint8_t Data, float *ToneLength);
22
23 void Performance(uint8_t Data, uint8_t *CurrentBar, uint8_t *NoteIndex, uint32_t *
    StartTime, uint16_t *Interval, float ToneLength, Pfm *Score);
24 #endif

```

A.2.11 カスタネットクライアントの関数実装 (castanets_client_function.cpp)

カスタネット用のクライアントの処理を実装したファイルである。Parse_data および BPM_update の処理は共通版と同様である。Performance 関数では、音の強さを送信せず、周波数と発音間隔のみをシリアル通信で Processing へ送信する。

```

1 #include "castanets_client_function.h"
2
3 bool Parse_data(uint8_t *Data, char Offset, WiFiUDP &Udp) {
4     int packetSize = Udp.parsePacket();
5
6     // 1. パケットを受信したかどうかの判定
7     if (packetSize) {
8         uint8_t received = Udp.read();
9         Udp.flush();
10
11         // === 【生存確認用デバッグ】受信した生の数値をそのままProcessingへ送る ===
12         Serial.print("RAW_DATA:");
13         Serial.println(received);
14         // =====
15
16         // 2. 受信データが40以上であればBPM値と判定 (フラグONを返す)
17         if (received >= 40) {
18             *Data = received;
19             return true; // フラグON
20         }
21         // 3. 40未満であれば小節番号と判定 (オフセットを適用し, フラグOFFを返す)
22         else {
23             // サーバーの小節番号とオフセットを足す
24             int myBar = received + Offset;
25
26             // マイナスになる場合 (まだ自分の出番ではない待機期間)
27             if (myBar < 0) {

```



```

28     *Data = 255; // 待機状態を維持するための無効な値を入れる
29 } else {
30     *Data = myBar; // 自分の小節番号として適用
31 }
32 return false; // フラグOFF
33 }
34 }
35 return false;
36 }
37
38 void BPM_update(uint8_t *CurrentBPM, bool Flag, uint8_t Data, float *ToneLength) {
39     // フラグがOFF (小節番号) の場合は何もせず終了
40     if (!Flag) {
41         return;
42     }
43     // フラグがON (BPM値) の場合の更新処理
44     else {
45         *CurrentBPM = Data;
46
47         // 基準となる4分音符の長さをミリ秒単位で計算 (60000 / BPM)
48         *ToneLength = 60000.0 / (*CurrentBPM);
49
50         // 確認用のシリアル出力
51         Serial.print("【BPM同期】新しいBPM:");
52         Serial.print(*CurrentBPM);
53         Serial.print("_->4分音符の基準長さ(ms):");
54         Serial.println(*ToneLength);
55     }
56 }
57
58 void Performance(uint8_t Data, uint8_t *CurrentBar, uint8_t *NoteIndex, uint32_t *
    StartTime, uint16_t *Interval, float ToneLength, Pfm *Score) {
59
60     // 1. 新しい小節番号を受信した時のリセット処理
61     // (Dataが40未満で、かつ現在演奏中の小節と異なる場合)
62     if (Data < 40 && Data != *CurrentBar) {
63         *CurrentBar = Data; // 現在の小節番号を更新
64         *NoteIndex = 0; // 読み込む音符を最初(0番目)に戻す
65         *StartTime = millis(); // 演奏開始タイマーをリセット
66         *Interval = 0; // 即座に最初の音が鳴るようにIntervalを0にする
67     }
68
69     // フリーズ防止
70     if (*CurrentBar >= 40) {
71         return;
72     }
73

```

```

74 // 2. 音を鳴らすタイミング（時間が経過したか）の判定
75 if (millis() - *StartTime >= *Interval) {
76
77     // その小節にある音符の数(noteCount)に達するまで処理を行う
78     if (*NoteIndex < Score[*CurrentBar].noteCount) {
79
80         *StartTime = millis(); // 次の音の基準時間を現在時刻に更新
81
82         // 譜面配列から「音の高さ(pitch)」と「音の長さの割合(length)」を取得
83         int pitch = Score[*CurrentBar].notes[*NoteIndex].pitch;
84         float length = Score[*CurrentBar].notes[*NoteIndex].length;
85
86         // 次の音までの間隔 (Interval) を計算 (ミリ秒)
87         *Interval = ToneLength * length;
88
89         // PC (Processing) ヘシリアル通信で送信
90         // 形式: 「周波数, 鳴らすミリ秒」 (例: 262, 500)
91         Serial.print(pitch);
92         Serial.print(",");
93         Serial.println(*Interval);
94
95         *NoteIndex = *NoteIndex + 1; // 次の音符へ進む
96     }
97 }
98 }

```

A.2.12 ピアノの音源 (piano.pde)

木村が担当したピアノの音源のソースコードである。PianoSound クラスが音源の中心であり、基本となる音、高域の輝きを与える音、および余韻を表す音の3種類のオシレータ群を重ね合わせることで、ピアノの音色を合成する。serialEvent() 関数でクライアントから受信した音符の情報を解釈して発音する。

```

1 import processing.serial.*;
2 import ddf.minim.*;
3 import ddf.minim.ugens.*;
4
5 // =====
6 // シリアル通信
7 // =====
8 Serial myPort;
9
10 // 前に確認したArduinoのポート番号
11 // [2] "/dev/cu.usbmodem34B7DA6537F82"
12 final int PORT_INDEX = 2;
13
14 // Arduino側が Serial.begin(115200); なので合わせる

```

```

15 final int BAUD_RATE = 115200;
16
17 // =====
18 // Minim
19 // =====
20 Minim minim;
21 AudioOutput out;
22
23 // =====
24 // 現在の状態
25 // =====
26 String currentNote = "REST";
27 float currentFreq = 0;
28 int currentDurationMs = 500;
29 int currentBpm = 120;
30 String lastReceived = "";
31
32 // =====
33 // テスト再生用
34 // =====
35 boolean testPlaying = false;
36 int testIndex = 0;
37 int nextNoteTime = 0;
38
39 float[] testFreqs = {
40     262, 294, 330, 349,
41     330, 294, 262, 0,
42     330, 349, 392, 440,
43     392, 349, 330, 0,
44     262, 262, 262, 262,
45     262, 294, 330, 349,
46     330, 294, 262, 0
47 };
48
49
50 // =====
51 // setup
52 // =====
53 void setup() {
54     size(600, 300);
55
56     minim = new Minim(this);
57     out = minim.getLineOut(Minim.MONO, 2048);
58
59     println("==_Serial_Port_List_==");
60     printArray(Serial.list());
61
62     if (Serial.list().length > PORT_INDEX) {

```

```

63     String portName = Serial.list()[PORT_INDEX];
64     println("接続ポート:_" + portName);
65
66     myPort = new Serial(this, portName, BAUD_RATE);
67     myPort.bufferUntil('\n');
68     myPort.clear();
69 } else {
70     println("注意:_PORT_INDEX_が範囲外です. ");
71     println("Serial.list()_の番号を確認してください. ");
72     println("pキーによるProcessing単体のテスト再生はできます. ");
73 }
74
75 textSize(24);
76
77 println("Processing起動完了");
78 println("受信形式: 周波数, 長さms");
79 println("例: 262, 500");
80 println("休符: 0, 500");
81 println("pキー: Processing単体でテスト再生");
82 }
83
84
85 // =====
86 // draw
87 // =====
88 void draw() {
89     background(255);
90
91     fill(0);
92     textSize(24);
93     text("Piano_Part", 230, 70);
94
95     textSize(18);
96     text("Press_P_to_test", 220, 115);
97     text("Note:_" + currentNote, 220, 155);
98     text("Freq:_" + currentFreq + "_Hz", 220, 185);
99     text("Duration:_" + currentDurationMs + "_ms", 180, 215);
100    text("BPM:_" + currentBpm, 245, 245);
101
102    textSize(12);
103    text("Last:_" + lastReceived, 80, 280);
104
105    // Pキーによるテスト再生
106    if (testPlaying && millis() >= nextNoteTime) {
107        playTestNote();
108
109        float beatLength = 60.0 / currentBpm;
110        nextNoteTime = millis() + int(beatLength * 1000);

```

```

111
112     testIndex++;
113 }
114
115 if (testPlaying && testIndex >= testFreqs.length) {
116     testPlaying = false;
117     currentNote = "REST";
118     currentFreq = 0;
119 }
120 }
121
122
123 // =====
124 // keyPressed
125 // =====
126 void keyPressed() {
127     if (key == 'p' || key == 'P') {
128         testPlaying = true;
129         testIndex = 0;
130         nextNoteTime = millis();
131         println("テスト再生開始");
132     }
133 }
134
135
136 // =====
137 // テスト再生
138 // =====
139 void playTestNote() {
140     if (testIndex < testFreqs.length) {
141         float freq = testFreqs[testIndex];
142
143         currentFreq = freq;
144         currentNote = freqToName(freq);
145
146         if (freq > 0) {
147             float beatLength = 60.0 / currentBpm;
148
149             // 透明感ある鍵盤音版
150             out.playNote(0, beatLength, new PianoSound(freq, 0.50, beatLength));
151         }
152     }
153 }
154
155
156 // =====
157 // Arduinoから受信したときに呼ばれる
158 //

```

```

159 // 今のArduino側の基本形式：
160 // 周波数,長さms
161 //
162 // 例：
163 // 262,500
164 // 294,500
165 // 0,500
166 //
167 // 0は休符
168 // =====
169 void serialEvent(Serial p) {
170     String data = p.readStringUntil('\n');
171
172     if (data == null) {
173         return;
174     }
175
176     data = trim(data);
177
178     if (data.length() == 0) {
179         return;
180     }
181
182     lastReceived = data;
183     println("受信:_" + data);
184
185     // Arduino側のデバッグ出力は演奏データではないので無視
186     if (data.startsWith("RAW_DATA:")) {
187         println("デバッグデータなので無視:_" + data);
188         return;
189     }
190
191     // Wi-Fi接続メッセージなど、カンマがない行は無視
192     if (data.indexOf(",") == -1) {
193         println("演奏データではないため無視:_" + data);
194         return;
195     }
196
197     String[] values = split(data, ',');
198
199     // =====
200     // 今のArduino側の形式：周波数,長さms
201     // 例：262,500
202     // =====
203     if (values.length == 2) {
204         float freq = 0;
205         int durationMs = 500;
206

```

```

207     try {
208         freq = float(trim(values[0]));
209     }
210     catch (Exception e) {
211         println("周波数の読み取りに失敗:_" + values[0]);
212         return;
213     }
214
215     try {
216         durationMs = int(trim(values[1]));
217     }
218     catch (Exception e) {
219         println("長さmsの読み取りに失敗:_" + values[1]);
220         durationMs = 500;
221     }
222
223     currentFreq = freq;
224     currentDurationMs = durationMs;
225     currentNote = freqToName(freq);
226
227     playFrequency(freq, durationMs);
228     return;
229 }
230
231 // =====
232 // 将来用：周波数,長さms,BPM
233 // 例：262,500,120
234 // =====
235 if (values.length >= 3) {
236     float freq = 0;
237     int durationMs = 500;
238     int bpm = currentBpm;
239
240     try {
241         freq = float(trim(values[0]));
242     }
243     catch (Exception e) {
244         println("周波数の読み取りに失敗:_" + values[0]);
245         return;
246     }
247
248     try {
249         durationMs = int(trim(values[1]));
250     }
251     catch (Exception e) {
252         println("長さmsの読み取りに失敗:_" + values[1]);
253         durationMs = 500;
254     }

```

```

255
256     try {
257         bpm = int(trim(values[2]));
258     }
259     catch (Exception e) {
260         println("BPMの読み取りに失敗:_" + values[2]);
261         bpm = currentBpm;
262     }
263
264     currentFreq = freq;
265     currentDurationMs = durationMs;
266     currentBpm = bpm;
267     currentNote = freqToName(freq);
268
269     playFrequency(freq, durationMs);
270     return;
271 }
272
273 println("形式が不明なため無視:_" + data);
274 }
275
276
277 // =====
278 // 周波数を鳴らす
279 // =====
280 void playFrequency(float freq, int durationMs) {
281     if (freq <= 0) {
282         println("休符");
283         currentNote = "REST";
284         return;
285     }
286
287     float durationSec = durationMs / 1000.0;
288
289     println("再生:_" + freq + "_Hz/__" + durationMs + "_ms");
290
291     // 透明感ある鍵盤音版
292     out.playNote(0, durationSec, new PianoSound(freq, 0.50, durationSec));
293 }
294
295
296
297
298 // =====
299 // 周波数を音名に変換
300 // =====
301 String freqToName(float freq) {
302     if (freq <= 0) {

```



```

303     return "REST";
304 }
305
306 if (abs(freq - 262) < 3) return "C4";
307 if (abs(freq - 277) < 3) return "C#4";
308 if (abs(freq - 294) < 3) return "D4";
309 if (abs(freq - 311) < 3) return "D#4";
310 if (abs(freq - 330) < 3) return "E4";
311 if (abs(freq - 349) < 3) return "F4";
312 if (abs(freq - 370) < 3) return "F#4";
313 if (abs(freq - 392) < 3) return "G4";
314 if (abs(freq - 415) < 3) return "G#4";
315 if (abs(freq - 440) < 3) return "A4";
316 if (abs(freq - 466) < 3) return "A#4";
317 if (abs(freq - 494) < 3) return "B4";
318 if (abs(freq - 523) < 3) return "C5";
319 if (abs(freq - 587) < 3) return "D5";
320 if (abs(freq - 659) < 3) return "E5";
321 if (abs(freq - 698) < 3) return "F5";
322 if (abs(freq - 784) < 3) return "G5";
323 if (abs(freq - 880) < 3) return "A5";
324
325 return nf(freq, 0, 1) + "Hz";
326 }
327
328 // =====
329 // Piano sound ピアノ感強化版 V2
330 //
331 // 前回より大きく変更した点：
332 // ・整数倍音ではなく、ピアノ弦っぽい少しズレた倍音にする
333 // ・打鍵直後だけ「コツン」と鳴る高い成分を足す
334 // ・高い倍音ほど急速に消えるようにする
335 // ・木の箱っぽい共鳴成分を少しだけ足す
336 //
337 // 注意：Processing の合成音だけなので、本物の録音サンプルほどにはならない。
338 // ただし、前回より「ピアノらしい減衰」と「打鍵感」は強くなる。
339 // =====
340 class PianoSound implements Instrument {
341     Oscil[] toneWaves;
342     ADSR[] toneEnvs;
343
344     Oscil[] shineWaves;
345     ADSR[] shineEnvs;
346
347     Oscil[] tailWaves;
348     ADSR[] tailEnvs;
349
350     float freq;

```

```

351 float amp;
352 float durationSec;
353
354 final int NUM_TONES = 10;
355 final int NUM_SHINE = 5;
356 final int NUM_TAIL = 3;
357
358 PianoSound(float freq, float amp, float durationSec) {
359     this.freq = freq;
360     this.amp = amp;
361     this.durationSec = durationSec;
362
363     toneWaves = new Oscil[NUM_TONES];
364     toneEnvs = new ADSR[NUM_TONES];
365
366     shineWaves = new Oscil[NUM_SHINE];
367     shineEnvs = new ADSR[NUM_SHINE];
368
369     tailWaves = new Oscil[NUM_TAIL];
370     tailEnvs = new ADSR[NUM_TAIL];
371
372     // -----
373     // 透明感ある鍵盤音版
374     // -----
375     // 前回の「ハンマー感」「箱鳴り」を弱める.
376     // 機械音っぽさを減らすため、音の立ち上がりを少し丸くし、
377     // 高い倍音は薄く短く入れる.
378
379     // 全体音量. 大きいとすぐ音割れして機械音になるので控えめ.
380     float noteAmp = amp * 0.44;
381     if (freq >= 500) noteAmp *= 0.88;
382     if (freq >= 700) noteAmp *= 0.82;
383
384     // 高音ほど高倍音を減らす. キンキン感を抑えるため.
385     float highScale = map(freq, 262, 880, 0.88, 0.28);
386     highScale = constrain(highScale, 0.28, 0.88);
387
388     // 低音は少しだけ太くする. ただし濁らない程度.
389     float lowScale = map(freq, 262, 880, 1.08, 0.82);
390     lowScale = constrain(lowScale, 0.82, 1.08);
391
392     // 3本弦っぽい薄い揺れ + 少しだけズレた倍音.
393     // 完全な整数倍音だと電子音っぽくなるので、ほんの少しだけズラす.
394     float[] toneRatio = {
395         0.9992,
396         1.0000,
397         1.0008,
398         2.004,

```

```

399     3.010,
400     4.018,
401     5.030,
402     6.045,
403     8.070,
404     10.100
405 };
406
407 // メイン成分.
408 // 高い倍音を入れすぎないことで透明感を出す.
409 float[] toneAmp = {
410     noteAmp * 0.20 * lowScale,
411     noteAmp * 0.34 * lowScale,
412     noteAmp * 0.18 * lowScale,
413     noteAmp * 0.100 * highScale,
414     noteAmp * 0.055 * highScale,
415     noteAmp * 0.030 * highScale,
416     noteAmp * 0.018 * highScale,
417     noteAmp * 0.011 * highScale,
418     noteAmp * 0.006 * highScale,
419     noteAmp * 0.0035 * highScale
420 };
421
422 // 打鍵音を丸くするため, Attackは前回より少し遅め.
423 float attackTime = map(freq, 262, 880, 0.008, 0.004);
424 attackTime = constrain(attackTime, 0.004, 0.008);
425
426 // 透明感を残すため, 低い成分は少し長め, 高い成分は短め.
427 float baseDecay = constrain(durationSec * 1.05, 0.22, 0.95);
428
429 float[] toneDecay = {
430     baseDecay * 1.12,
431     baseDecay * 1.22,
432     baseDecay * 1.08,
433     baseDecay * 0.52,
434     baseDecay * 0.36,
435     baseDecay * 0.25,
436     baseDecay * 0.18,
437     baseDecay * 0.13,
438     baseDecay * 0.09,
439     baseDecay * 0.065
440 };
441
442 float[] toneSustain = {
443     0.0030,
444     0.0035,
445     0.0028,
446     0.00070,

```

```

447     0.00045,
448     0.00028,
449     0.00018,
450     0.00010,
451     0.00006,
452     0.00000
453 };
454
455 // 余韻. 短すぎると電子音のブツ切れ感が出るので少し長め.
456 float releaseTime = map(freq, 262, 880, 0.34, 0.14);
457 releaseTime = constrain(releaseTime, 0.14, 0.34);
458
459 for (int i = 0; i < NUM_TONES; i++) {
460     // 透明感重視なので基本はSINEだけ. TRIANGLEは機械音感が出やすいので使わない.
461     toneWaves[i] = new Oscil(freq * toneRatio[i], 1.0, Waves.SINE);
462
463     toneEnvs[i] = new ADSR(
464         toneAmp[i],
465         attackTime,
466         toneDecay[i],
467         toneSustain[i],
468         releaseTime
469     );
470
471     toneWaves[i].patch(toneEnvs[i]);
472 }
473
474 // -----
475 // きらっとした透明感成分
476 // -----
477 // 打鍵音ではなく、ガラスっぽい薄い響き.
478 // 強すぎると鉄琴になるのでかなり小さくする.
479 float shineScale = map(freq, 262, 880, 0.70, 0.24);
480 shineScale = constrain(shineScale, 0.24, 0.70);
481
482 float[] shineRatio = {
483     2.414,
484     3.018,
485     5.027,
486     7.080,
487     11.120
488 };
489
490 float[] shineAmp = {
491     noteAmp * 0.013 * shineScale,
492     noteAmp * 0.010 * shineScale,
493     noteAmp * 0.0065 * shineScale,
494     noteAmp * 0.0036 * shineScale,

```

```

495     noteAmp * 0.0019 * shineScale
496 };
497
498 float[] shineDecay = {
499     0.115,
500     0.090,
501     0.065,
502     0.045,
503     0.032
504 };
505
506 for (int i = 0; i < NUM_SHINE; i++) {
507     shineWaves[i] = new Oscil(freq * shineRatio[i], 1.0, Waves.SINE);
508     shineEnvs[i] = new ADSR(
509         shineAmp[i],
510         0.006,
511         shineDecay[i],
512         0.0,
513         0.055
514     );
515     shineWaves[i].patch(shineEnvs[i]);
516 }
517
518 // -----
519 // 薄い余韻成分
520 // -----
521 // リバーブの代わりに、かなり小さい音量で長く残る成分を足す。
522 // これで音の終わりが少し空気っぽくなる。
523 float tailScale = map(freq, 262, 880, 0.75, 0.35);
524 tailScale = constrain(tailScale, 0.35, 0.75);
525
526 float[] tailRatio = {
527     1.0000,
528     2.0020,
529     3.0060
530 };
531
532 float[] tailAmp = {
533     noteAmp * 0.030 * tailScale,
534     noteAmp * 0.012 * tailScale,
535     noteAmp * 0.006 * tailScale
536 };
537
538 for (int i = 0; i < NUM_TAIL; i++) {
539     tailWaves[i] = new Oscil(freq * tailRatio[i], 1.0, Waves.SINE);
540     tailEnvs[i] = new ADSR(
541         tailAmp[i],
542         0.020,

```

```

543         constrain(durationSec * 1.20, 0.30, 1.10),
544         0.0,
545         releaseTime * 1.25
546     );
547     tailWaves[i].patch(tailEnvs[i]);
548 }
549 }
550
551 void noteOn(float dur) {
552     for (int i = 0; i < NUM_TONES; i++) {
553         toneEnvs[i].patch(out);
554         toneEnvs[i].noteOn();
555     }
556
557     for (int i = 0; i < NUM_SHINE; i++) {
558         shineEnvs[i].patch(out);
559         shineEnvs[i].noteOn();
560     }
561
562     for (int i = 0; i < NUM_TAIL; i++) {
563         tailEnvs[i].patch(out);
564         tailEnvs[i].noteOn();
565     }
566 }
567
568 void noteOff() {
569     for (int i = 0; i < NUM_TONES; i++) {
570         toneEnvs[i].noteOff();
571         toneEnvs[i].unpatchAfterRelease(out);
572     }
573
574     for (int i = 0; i < NUM_SHINE; i++) {
575         shineEnvs[i].noteOff();
576         shineEnvs[i].unpatchAfterRelease(out);
577     }
578
579     for (int i = 0; i < NUM_TAIL; i++) {
580         tailEnvs[i].noteOff();
581         tailEnvs[i].unpatchAfterRelease(out);
582     }
583 }
584 }

```

A.2.13 ヴァイオリンの音源 (violin.pde)

仲里が担当したヴァイオリンの音源のソースコードである。ViolinInstrument クラスが音源の中心であり、胴の響きを表す波形、弦の成分を表す2つののこぎり波、およびビブラートを与え

るオシレータを組み合わせることで、ヴァイオリンの音色を合成する。serialEvent() 関数でクライアントから受信した音符の情報を解釈して発音する。

```
1 // =====
2 // 「かえるのうた」 ヴァイオリン・パート (Processing側) — 最小表示版 v4
3 // =====
4 // クライアントArduino (violin_client_function.cpp の Performance()) から
5 //   "周波数, 鳴らすミリ秒, 弾き方\n"   例) 392.00, 500, 0
6 //   を受信し、ヴァイオリン音色で演奏します。
7 //
8 //   弾き方の値 (Arduino から届く数値) :
9 //     0 = 下げ弓 (しっかりした弓のストローク)
10 //     1 = 上げ弓 (軽い弓のストローク)   ← 0と1の交互で「弓の上下」を表現
11 //     2 = デタシェ (テスト用)
12 //   ※「クゥ」も「ケケケ」も、譜面側で 0/1 を交互に並べてあるので、
13 //   下げ弓・上げ弓が交互に鳴り、弓を上下に返す質感になります。
14 //   ※音符の長さに応じて立ち上がりの鋭さを自動調整 (速い刻みは鋭く、
15 //   ゆっくりした旋律は表情豊かに)。
16 //
17 //   【必要なライブラリ】 Minim
18 //   【セットアップ】 portName を自分の環境のポート名に書き換える。
19 //   【Arduino無しのテスト】
20 //     1~6 = ド〜ラ (押すたびに下げ弓→上げ弓を交互), 0 = 休符,
21 //     t = 「ケケケケ」走句 (下げ/上げ交互) を自動で鳴らす。
22 // =====
23
24 import processing.serial.*;
25 import ddf.minim.*;
26 import ddf.minim.ugens.*;
27
28 Minim minim;
29 AudioOutput out;
30 Serial port;
31
32 Wavetable customViolinWave;
33
34 // ★自分の環境のポート名に書き換えてください
35 String portName = "/dev/cu.usbmodem34B7DA6377FC2";
36
37 // ---- 表示用の状態 ----
38 String currentNoteName    = "—";
39 boolean currentIsRest     = true;
40 boolean hasReceivedAnyData = false;
41
42 // ---- 受信ログ (発音数の検証用CSV) ----
43 //   起動時に violin_recv_log.csv を作成し、受信した音符を1行ずつ記録する。
44 //   k キーで保存して閉じる (閉じなくても1行ごとに書き込まれる)。
45 PrintWriter recvLog;
```

```

46 int recvCount = 0;
47
48 // =====
49 // ★音色つまみ（演奏を聞きながらキーで調整できる）
50 //   金管/ポオー → 弦らしく したいときの目安：
51 //     ・ 暗くこもる(ポオー) → gBright を上げる(q)
52 //     ・ 弦の擦れ感が欲しい → gBowNoise を上げる(w)
53 //     ・ 金管っぽい鳴き      → gReso を下げる(d)
54 //     ・ 丸い/ホルンっぽい   → gSaw を上げ gBody を下げる(r)
55 //   キー： q/a=明るさ w/s=擦れ音 e/d=共振 r/f=ノコギリ⇄胴
56 //   現在値はコンソールと画面左上に表示。良い値が決まったら教えてください。
57 // =====
58 float gBright = 4000; // フィルタ明るさ(Hz)。低いと倍音が消えて純音(ポオー)化する
59 float gReso    = 0.0; // フィルタ共振。高い=金管っぽい鳴き/0=素直
60 float gBowNoise = 0.22; // 弓の擦れ音の量。高い=弦らしいザラつき
61 float gSaw      = 0.60; // ノコギリ波(弦成分=倍音が豊かな)の量
62 float gBody     = 0.20; // 胴成分(丸み・純音寄り)の量
63
64 void setup() {
65     size(800, 400);
66
67     minim = new Minim(this);
68     out = minim.getLineOut();
69
70     PFont jpFont = findJapaneseFont(28);
71     if (jpFont != null) textFont(jpFont);
72
73     // ヴァイオリンらしい倍音波形 (violin.pde と同じ作り方)
74     float[] harmonics = {0.8729, 1.0000, 0.1919, 0.1815, 0.1828, 0.0367, 0.0338,
75                          0.0533, 0.0968, 0.0600};
76     float[] waveData = new float[1024];
77     float maxVal = 0;
78     for (int i = 0; i < 1024; i++) {
79         float t = (float)i / 1024.0 * TWO_PI;
80         float val = 0;
81         for (int h = 0; h < harmonics.length; h++) val += harmonics[h] * sin((h + 1) * t);
82
83         waveData[i] = val;
84         if (abs(val) > maxVal) maxVal = abs(val);
85     }
86     for (int i = 0; i < 1024; i++) waveData[i] /= maxVal;
87     customViolinWave = new Wavetable(waveData);
88
89     // 受信ログCSVを作成（ヘッダ行を書く）
90     recvLog = createWriter("violin_recv_log.csv");
91     recvLog.println("番号,周波数Hz,長さms,弾き方,音名,種別");
92     recvLog.flush();
93 }

```



```

92     println("利用可能なシリアルポート一覧:");
93     printArray(Serial.list());
94     openSerialPort();
95     println("「かえるのうた」 ヴァイオリン・パート_準備完了。");
96 }
97
98 void openSerialPort() {
99     try {
100         port = new Serial(this, portName, 115200);
101     } catch (RuntimeException e) {
102         println("ポート_" + portName + "_を開けませんでした:_" + e.getMessage());
103         String[] avail = Serial.list();
104         if (avail.length > 0) {
105             println("代わりに先頭のポート_" + avail[0] + "_を使用します。");
106             port = new Serial(this, avail[0], 115200);
107         } else {
108             println("利用可能なシリアルポートが見つかりません。");
109             return;
110         }
111     }
112     port.bufferUntil('\n');
113 }
114
115 PFont findJapaneseFont(int fontSize) {
116     String[] candidates = {
117         "Hiragino_Sans", "Hiragino_Kaku_Gothic_ProN", "Yu_Gothic", "YuGothic",
118         "Meiryō", "MS_Gothic", "MS_PGothic", "Noto_Sans_CJK_JP", "Noto_Sans_JP",
119         "Source_Han_Sans_JP", "IPAGothic", "TakaoGothic"
120     };
121     String[] available = PFont.list();
122     for (String cand : candidates) {
123         for (String avail : available) {
124             if (avail.toLowerCase().contains(cand.toLowerCase())) {
125                 return createFont(avail, fontSize, true);
126             }
127         }
128     }
129     return null;
130 }
131
132 // =====
133 // 描画：波形（上半分） + 音階（下半分）だけの最小構成
134 // =====
135 void draw() {
136     background(20, 30, 40);
137
138     updateTestRun(); // テスト走句スケジューラ
139

```

```

140 // --- 1. 波形（オシロスコープ）：中心Y=100 ---
141 stroke(150, 255, 150);
142 strokeWeight(2);
143 noFill();
144 beginShape();
145 for (int i = 0; i < out.bufferSize(); i++) {
146     float x = map(i, 0, out.bufferSize() - 1, 0, width);
147     vertex(x, 100 + out.left.get(i) * 100);
148 }
149 endShape();
150 strokeWeight(1);
151
152 // --- 2. 音階：画面下半分に大きく表示 ---
153 textAlign(CENTER, CENTER);
154 fill(currentIsRest ? color(120, 140, 130) : color(255, 244, 214));
155 textSize(120);
156 String shown = hasReceivedAnyData ? currentNoteName : "—";
157 text(shown, width / 2, 290);
158
159 // --- 3. 音色つまみの現在値（左上） ---
160 textAlign(LEFT, TOP);
161 fill(180, 200, 220);
162 textSize(13);
163 text("q/a_明るさ:" + int(gBright)
164     + "_w/s_擦れ:" + nf(gBowNoise, 1, 2)
165     + "_e/d_共振:" + nf(gReso, 1, 2)
166     + "_r/f_ノコギリ:" + nf(gSaw, 1, 2) + "/胴:" + nf(gBody, 1, 2),
167     10, 8);
168 }
169
170 // =====
171 // シリアル通信：3つのデータを受け取る
172 // =====
173 void serialEvent(Serial p) {
174     String inString = p.readStringUntil('\n');
175     if (inString == null) return;
176     inString = trim(inString);
177     String[] data = split(inString, ',');
178     if (data.length != 3) return; // RAW_DATA: 等のデバッグ行は無視
179
180     float freq = float(data[0]);
181     float durationMs = float(data[1]);
182     int articulation = int(data[2]);
183
184     // 受信した音符をCSVに記録（発音数の検証用）
185     recvCount++;
186     String shubetsu = (freq > 0) ? "発音" : "休符";
187     recvLog.println(recvCount + "," + int(freq) + "," + int(durationMs) + ",");

```

```

188         + articulation + "," + noteNameFor(freq) + "," + shubetsu);
189     recvLog.flush();
190
191     triggerNote(freq, durationMs, articulation);
192 }
193
194 void triggerNote(float freq, float durationMs, int articulation) {
195     currentNoteName = noteNameFor(freq);
196     currentIsRest = (freq <= 0);
197     hasReceivedAnyData = true;
198
199     if (freq > 0) {
200         float durationSec = durationMs / 1000.0;
201         float playDuration = durationSec * 0.85;
202         if (playDuration < 0.05) playDuration = 0.05;
203         // 音符の長さを合成側に渡し、立ち上がりの鋭さを自動調整
204         out.playNote(0, playDuration, new ViolinInstrument(freq, 0.8, articulation,
205             playDuration));
206     }
207 }
208
209 String noteNameFor(float freq) {
210     if (freq <= 0) return "休符";
211     float[] freqs = {262, 294, 330, 349, 392, 440, 523, 587, 659, 698, 784, 880};
212     String[] names = {"ド", "レ", "ミ", "ファ", "ソ", "ラ", "高ド", "高レ", "高ミ", "高
213         ファ", "高ソ", "高ラ"};
214     int best = 0;
215     float bestDiff = 999999.0;
216     for (int i = 0; i < freqs.length; i++) {
217         float diff = abs(freq - freqs[i]);
218         if (diff < bestDiff) { bestDiff = diff; best = i; }
219     }
220     return names[best];
221 }
222
223 // ---- テスト用 ----
224 int testBow = 0; // 0=下げ弓, 1=上げ弓 を交互に
225 float[] testRunFreqs;
226 int testRunIndex = -1;
227 int testRunNextMs = 0;
228 int testRunStepMs = 220;
229
230 void keyPressed() {
231     float[] tf = {262, 294, 330, 349, 392, 440};
232     if (key >= '1' && key <= '6') {
233         triggerNote(tf[key - '1'], 500, testBow);
234         testBow = 1 - testBow;
235     } else if (key == '0') {

```

```

234     triggerNote(0, 500, 0);
235 } else if (key == 't' || key == 'T') {
236     testRunFreqs = new float[] {262, 262, 294, 294, 330, 330, 349, 349};
237     testRunIndex = 0;
238     testRunNextMs = millis();
239 }
240 // ---- 音色つまみ（聞きながら調整） ----
241 else if (key == 'q') { gBright += 200; printTone(); }
242 else if (key == 'a') { gBright = max(500, gBright-200); printTone(); }
243 else if (key == 'w') { gBowNoise += 0.03; printTone(); }
244 else if (key == 's') { gBowNoise = max(0, gBowNoise-0.03); printTone(); }
245 else if (key == 'e') { gReso = min(0.6, gReso+0.03); printTone(); }
246 else if (key == 'd') { gReso = max(0, gReso-0.03); printTone(); }
247 else if (key == 'r') { gSaw += 0.05; gBody = max(0, gBody-0.05); printTone(); } //
    弦寄り
248 else if (key == 'f') { gSaw = max(0, gSaw-0.05); gBody += 0.05; printTone(); } //
    胴/丸み寄り
249 // ---- 受信ログの保存 ----
250 else if (key == 'k' || key == 'K') {
251     recvLog.flush();
252     println("受信ログ保存:_violin_recv_log.csv (" + recvCount + "_行)");
253 }
254 }
255
256 void printTone() {
257     println("[音色]_明るさ_gBright=" + int(gBright)
258         + "_共振_gReso=" + nf(gReso,1,2)
259         + "_擦れ_gBowNoise=" + nf(gBowNoise,1,2)
260         + "_ノコギリ_gSaw=" + nf(gSaw,1,2)
261         + "_胴_gBody=" + nf(gBody,1,2));
262 }
263
264 void updateTestRun() {
265     if (testRunIndex < 0) return;
266     if (millis() >= testRunNextMs) {
267         if (testRunIndex < testRunFreqs.length) {
268             // 下げ弓(0)→上げ弓(1)を交互に（実際の譜面と同じ）
269             triggerNote(testRunFreqs[testRunIndex], testRunStepMs, testRunIndex % 2);
270             testRunIndex++;
271             testRunNextMs += testRunStepMs;
272         } else {
273             testRunIndex = -1;
274         }
275     }
276 }
277
278 void stop() {
279     if (recvLog != null) { recvLog.flush(); recvLog.close(); }

```

```

280 out.close();
281 minim.stop();
282 super.stop();
283 }
284
285 // =====
286 // ヴァイオリンクラス (v4)
287 // ・ノコギリ波の太さを復活 (弦の胴鳴り重視)
288 // ・低い音 (クワのC4等) はフィルターを暖かい帯域まで下げて金管化を回避
289 // ・弓の擦れ音 (PINKノイズ) を全弓使いで維持して弦の質感を確保
290 // ・0=下げ弓 / 1=上げ弓 を弾き分け、交互で「弓の上下」を表現
291 // ・音符の長さに応じて立ち上がり/余韻を自動調整
292 // =====
293 class ViolinInstrument implements Instrument {
294     Oscil waveBody, waveString1, waveString2, vibrato;
295     Noise bowAttack, bowFriction;
296     ADSR attackEnv, frictionEnv, ampEnv;
297     Summer mix;
298     MoogFilter filter;
299     Line pitchSlide;
300
301     ViolinInstrument(float frequency, float amplitude, int art, float noteLenSec) {
302
303         // 短い音ほど立ち上がり・余韻を詰める係数 (速い刻みは鋭く、旋律は豊かに)
304         float lenF = constrain(noteLenSec / 0.45, 0.40, 1.25);
305
306         // 弦成分(ノコギリ gSaw) + 胴成分(丸み gBody)。つまみで比率を変えられる
307         waveBody = new Oscil(frequency, amplitude * gBody,
308                               customViolinWave);
309         waveString1 = new Oscil(frequency * 1.002, amplitude * gSaw * 0.5, Waves.SAW);
310         waveString2 = new Oscil(frequency * 0.998, amplitude * gSaw * 0.5, Waves.SAW);
311
312         // ピッチのしゃくり (下げ弓は強め、上げ弓は軽め)
313         float slideTime, slideStart;
314         if (art == 1) { // 上げ弓
315             slideTime = 0.05 * lenF; slideStart = frequency - 7;
316         } else if (art == 2) { // デタシェ
317             slideTime = 0.03 * lenF; slideStart = frequency - 5;
318         } else { // 下げ弓
319             slideTime = 0.09 * lenF; slideStart = frequency - 11;
320         }
321         pitchSlide = new Line(slideTime, slideStart, frequency);
322
323         vibrato = new Oscil(5.5, frequency * 0.012, Waves.SINE);
324         pitchSlide.patch(vibrato.offset);
325         vibrato.patch(waveBody.frequency);
326         vibrato.patch(waveString1.frequency);
327         vibrato.patch(waveString2.frequency);

```

```

327
328 mix = new Summer();
329 waveBody.patch(mix);
330 waveString1.patch(mix);
331 waveString2.patch(mix);
332
333 // フィルタ（つまみ：明るさ gBright・共振 gReso）。固定（スweep無し）
334 filter = new MoogFilter(gBright, gReso, MoogFilter.Type.LP);
335 mix.patch(filter);
336
337 // 弓ノイズ・エンベロープ（弾き方ごと）。擦れ音の量は つまみ gBowNoise
338 float attackNoiseAmp, frictionNoiseAmp;
339 if (art == 1) { // 上げ弓（軽い）
340     attackNoiseAmp = amplitude * 0.30;
341     frictionNoiseAmp = amplitude * gBowNoise;
342     attackEnv = new ADSR(1.0, 0.016 * lenF, 0.04, 0.0, 0.07 * lenF);
343     frictionEnv = new ADSR(1.0, 0.025, 0.05, 0.85, 0.09 * lenF);
344     ampEnv = new ADSR(1.0, 0.022 * lenF, 0.06, 0.82, 0.10 * lenF);
345 } else if (art == 2) { // デタシェ
346     attackNoiseAmp = amplitude * 0.34;
347     frictionNoiseAmp = amplitude * gBowNoise;
348     attackEnv = new ADSR(1.0, 0.010 * lenF, 0.03, 0.0, 0.05 * lenF);
349     frictionEnv = new ADSR(1.0, 0.015, 0.03, 0.90, 0.06 * lenF);
350     ampEnv = new ADSR(1.0, 0.014 * lenF, 0.04, 0.80, 0.06 * lenF);
351 } else { // 下げ弓（しっかり）
352     attackNoiseAmp = amplitude * 0.38;
353     frictionNoiseAmp = amplitude * gBowNoise;
354     attackEnv = new ADSR(1.0, 0.020 * lenF, 0.05, 0.0, 0.10 * lenF);
355     frictionEnv = new ADSR(1.0, 0.045, 0.07, 0.88, 0.14 * lenF);
356     ampEnv = new ADSR(1.0, 0.040 * lenF, 0.08, 0.85, 0.16 * lenF);
357 }
358
359 bowAttack = new Noise(attackNoiseAmp, Noise.Tint.WHITE);
360 bowFriction = new Noise(frictionNoiseAmp, Noise.Tint.PINK); // 弓の擦れ音
361
362 bowAttack.patch(attackEnv);
363 attackEnv.patch(mix);
364 bowFriction.patch(frictionEnv);
365 frictionEnv.patch(mix);
366 filter.patch(ampEnv);
367 }
368
369 void noteOn(float duration) {
370     ampEnv.noteOn();
371     attackEnv.noteOn();
372     frictionEnv.noteOn();
373     pitchSlide.activate();
374     ampEnv.patch(out);

```

```
375     }  
376  
377     void noteOff() {  
378         ampEnv.noteOff();  
379         attackEnv.noteOff();  
380         frictionEnv.noteOff();  
381         ampEnv.unpatchAfterRelease(out);  
382     }  
383 }
```